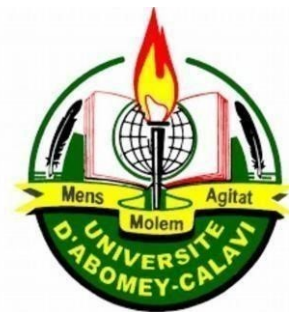


REPUBLIQUE DU BENIN



MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA
RECHERCHE SCIENTIFIQUE

DIRECTION GENERALE DE L'ENSEIGNEMENT SUPERIEUR

UNIVERSITE D'ABOMEY CALAVI

ECOLE POLYTECHNIQUE D'ABOMEY-CALAVI

DEPARTEMENT DE GENIE ELECTRIQUE ET GENIE INFORMATIQUE ET
TELECOMMUNICATION

THEME : Système de gestion scolaire

Groupe 5 GE

Produit par:

ATCHOBELI Romilus

KPOKPO Sunday

KINKINGNINHOUN Johannes

Sous la supervision de :

Dr SOGBOHOSSOU Médésu

Année académique : 2025-2026

PLAN

I- Cahier des charges

1. Introduction Générale
2. Description des acteurs
3. Description des fonctionnalités
4. Contraintes et Exigences

II- Diagramme des cas d'utilisation

III- Représentation textuelle des cas d'utilisation

IV- Identification des classes d'objets, leurs opérations et attributs

V- Diagrammes de séquence

VI- Diagramme de classe

VII- Conception des écrans avec wxSmith

VIII- Conclusion

I- CAHIER DES CHARGES

Système de Gestion Scolaire - EPAC (GE & GIT)

1. INTRODUCTION GÉNÉRALE

1.1 Contexte

L'École Polytechnique d'Abomey-Calavi (EPAC) forme des ingénieurs de haut niveau dans plusieurs spécialités, notamment en Génie Électrique (GE) et en Génie Informatique et Télécommunications (GIT). La première année du cycle ingénieur (niveau L3) concentre un effectif important d'étudiants suivant un programme académique dense avec de nombreux cours.

Actuellement, la gestion administrative de ces étudiants s'effectue de manière partiellement manuelle, entraînant plusieurs difficultés :

- Erreurs lors de la saisie des notes et calculs de moyennes
- Difficulté de suivi rigoureux des absences
- Génération laborieuse des emplois du temps
- Manque de traçabilité des données académiques
- Accès limité aux informations pour les étudiants et enseignants

1.2 Objectifs du projet

Ce projet vise à concevoir un système informatisé permettant de :

Objectif principal : Automatiser et optimiser la gestion académique des étudiants L3 des départements GE et GIT de l'EPAC.

Objectifs spécifiques :

- Centraliser les informations des étudiants et enseignants dans une base de données unique
- Automatiser le calcul des moyennes selon les pondérations définies
- Faciliter la génération et la consultation des emplois du temps
- Assurer un suivi rigoureux des absences avec alertes automatiques au seuil de 20%
- Améliorer l'accessibilité aux données académiques pour tous les acteurs
- Réduire les erreurs de saisie et améliorer la traçabilité
- Permettre la consultation des notes en temps réel dès leur saisie
- Offrir plusieurs formats de bulletin (PDF, email, visualisation en ligne)
- Notifier automatiquement les étudiants lors de la disponibilité de leurs bulletins

1.3 Périmètre du système

Inclus :

- Gestion des étudiants niveau L3 uniquement
- Départements GE et GIT
- Calcul automatique des notes
- Suivi des absences
- Génération d'emplois du temps

Exclus :

- Gestion financière (frais de scolarité)
- Autres niveaux (L1, L2, M1, M2)
- Gestion de bibliothèque
- Système de messagerie
- Plateforme e-learning

2. DESCRIPTION DES ACTEURS

2.1 Administrateur

Rôle : Gestion complète du système

Responsabilités :

- Créer, modifier et supprimer les comptes étudiants et enseignants
- Gérer le catalogue des cours (création, modification, suppression)
- Affecter les enseignants aux cours
- Valider les inscriptions des étudiants (les étudiants s'inscrivent eux-mêmes)
- Créer et gérer les emplois du temps
- Vérifier et valider les notes saisies par les enseignants
- Consulter toutes les statistiques et rapports (notes, absences, inscriptions)
- Administrer les droits d'accès Contraintes :
- Accès sécurisé par identifiant et mot de passe
- Responsable de la cohérence des données
- N'enregistre pas directement les absences (réservé aux enseignants)

2.2 Enseignant

Rôle : Gestion pédagogique de ses cours

Responsabilités :

- Consulter la liste de ses cours assignés

Consulter la liste des étudiants inscrits à ses cours

- Saisir les notes (Contrôle Continu, Travaux Pratiques, Examen)
- Enregistrer les absences lors de ses séances
- Consulter son emploi du temps personnel
- Générer des relevés de notes pour ses cours
- Consulter les statistiques de ses cours (taux de réussite, absences)

Contraintes :

- Ne peut modifier que les notes de ses propres cours
- Ne peut pas modifier les informations des étudiants
- Ne peut pas consulter les notes des autres enseignants

2.3 Étudiant

Rôle : Consultation de son parcours académique

Responsabilités :

- S'inscrire pour l'année dans sa classe (inscription en attente de validation)
- Consulter ses informations personnelles
- Consulter la liste de ses cours du semestre
- Consulter ses notes (CC, TP, Examen, Moyenne)
- Télécharger/imprimer son relevé de notes
- Consulter son emploi du temps personnel
- Consulter ses absences par cours
- Recevoir des alertes en cas de dépassement du seuil d'absences
- Modifier ses informations de contact (email, téléphone) Contraintes :
- Accès en lecture seule (pas de modification des notes)
- Ne peut pas consulter les notes des autres étudiants

3. DESCRIPTION DES FONCTIONNALITÉS

3.1 Module Gestion des Étudiants

Fonctionnalités :

- Créer un étudiant : Enregistrer un nouvel étudiant avec ses informations (matricule, nom, prénom, date de naissance, département, niveau L3, email, téléphone)
- Modifier un étudiant : Mettre à jour les informations d'un étudiant existant
- Supprimer un étudiant : Retirer un étudiant du système

Rechercher un étudiant : Par matricule, nom ou département



- Lister les étudiants : Afficher tous les étudiants avec filtres (département, statut)
- Consulter le dossier étudiant : Voir toutes les informations académiques d'un étudiant

Acteurs concernés : Administrateur (toutes), Étudiant (consultation uniquement)

3.2 Module Gestion des Enseignants

Fonctionnalités :

- Créer un enseignant : Enregistrer un nouvel enseignant (nom, prénom, spécialité, grade, département, email, téléphone)
- Modifier un enseignant : Mettre à jour les informations
- Supprimer un enseignant : Retirer un enseignant du système
- Affecter un enseignant à un cours : Assigner un ou plusieurs cours à un enseignant
- Consulter la charge d'enseignement : Voir les cours et heures attribuées
- Lister les enseignants : Avec filtres par département ou spécialité

Acteurs concernés : Administrateur (toutes), Enseignant (consultation de ses propres infos)

3.3 Module Gestion des Cours Fonctionnalités

:

- Créer un cours : Définir un cours (code, intitulé, crédits ECTS, nombre d'heures, semestre, département)
- Modifier un cours : Mettre à jour les informations d'un cours
- Supprimer un cours : Retirer un cours du catalogue
- Affecter un enseignant : Assigner un responsable au cours
- Définir les modalités d'évaluation : Préciser les pondérations (CC, TP, Examen)
- Lister les cours : Par département, semestre ou enseignant

Acteurs concernés : Administrateur (toutes), Enseignant (consultation), Étudiant (consultation)

3.4 Module Gestion des Inscriptions

Fonctionnalités :

- S'inscrire pour l'année : L'étudiant s'inscrit pour l'année dans sa classe (automatiquement inscrit à tous les cours de son département et semestre)

- Valider les inscriptions : L'administrateur valide l'inscription administrative
Consulter les inscriptions : Voir la liste des étudiants inscrits par cours (avec recherche)
- Annuler une inscription : En cas d'erreur administrative

Acteurs concernés : Administrateur (validation), Enseignant (consultation), Étudiant (inscription et consultation)

3.5 Module Gestion des Notes Fonctionnalités

:

- Saisir les notes : Enregistrer les notes de CC, TP et Examen pour chaque étudiant
 - Calcul automatique de la moyenne par cours : Dès la saisie des notes
 - Calcul de la moyenne générale : Moyenne de tous les cours (pondérée par crédits ECTS)
 - Consultation en temps réel : L'étudiant voit ses notes immédiatement après saisie par l'enseignant
 - Modifier les notes : Correction d'erreurs de saisie par l'enseignant (avant validation finale)
 - Valider les notes : L'administrateur valide définitivement les notes d'un cours
 - Génération automatique du bulletin : Dès que toutes les notes du semestre sont validées, génération automatique du relevé officiel
 - Téléchargement multi-format du bulletin :
 - Format PDF (impression officielle)
 - Envoi automatique par email
 - Visualisation en ligne
 - Consulter les statistiques : Taux de réussite, moyenne de classe, distribution des notes par cours
 - Historique des modifications : Traçabilité de toutes les modifications de notes
- Acteurs concernés :
- Administrateur (vérification et validation des notes, consultation statistiques)
 - Enseignant (saisie et modification de ses notes, consultation)
 - Étudiant (consultation temps réel, téléchargement bulletin)
- Contraintes :
- Notes sur 20
 - Consultation en temps réel dès saisie (pas d'attente fin semestre)
 - Le bulletin n'est généré qu'après validation de TOUTES les notes du semestre
 - Notes modifiables uniquement avant validation par l'enseignant
 - Notification automatique à l'étudiant lors de la disponibilité du bulletin

3.6 Module Gestion de l'Emploi du Temps

Fonctionnalités :

- Créer un emploi du temps : Définir les créneaux horaires pour chaque cours
 - Affecter une salle : Assigner une salle de cours à un créneau
 - Gérer les créneaux : Jour, heure de début, heure de fin, durée
 - Éviter les conflits : Vérifier qu'un enseignant ou une salle ne soit pas doublebookée
 - Consulter l'emploi du temps :
 - Par étudiant : emploi du temps personnel
 - Par enseignant : emploi du temps personnel
 - Par salle : occupation de la salle
 - Par département : emploi du temps global
 - Exporter l'emploi du temps : Format PDF ou image
- Acteurs concernés : Administrateur (création), Enseignant (consultation), Étudiant (consultation) Contraintes :
- Durée variable selon le cours
 - Pas de cours pendant la pause déjeuner (12h30 – 14h00)

3.7 Module Gestion des Absences

Fonctionnalités :

- Enregistrer une absence : Marquer un étudiant absent à une séance de cours
 - Justifier une absence : Ajouter un motif et un justificatif
 - Calculer le taux d'absence : Pourcentage d'absences par cours pour chaque étudiant
 - Alerter en cas de dépassement : Notification automatique si taux > 20%
- Consulter les absences :
- Par étudiant : toutes ses absences
 - Par cours : liste des absents
 - Par date : absences du jour
- Générer des rapports d'assiduité : Statistiques par cours ou par étudiant Acteurs concernés :
- Enseignant (enregistrement et consultation des absences pour ses cours, génération rapports)
 - Étudiant (consultation de ses absences et justification)

4. CONTRAINTES ET EXIGENCES

4.1 Contraintes techniques

- Interface graphique ergonomique et intuitive
- Base de données relationnelle
- Sécurité : authentification et gestion des droits d'accès
- Sauvegarde régulière des données

4.2 Contraintes fonctionnelles

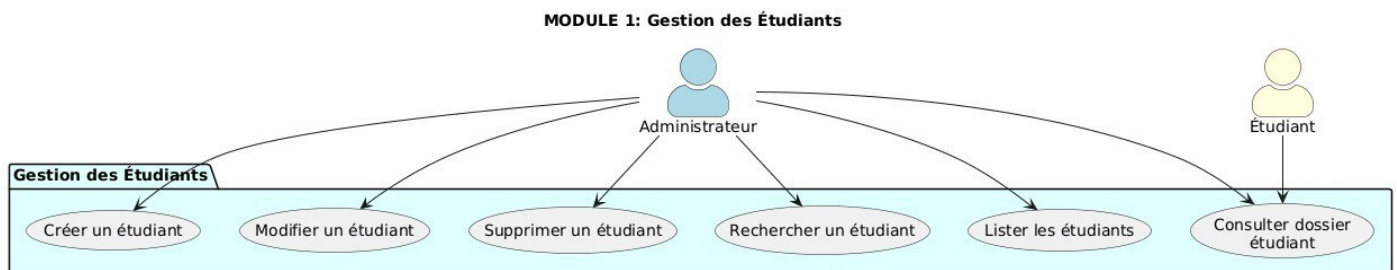
- Calcul automatique et instantané des moyennes
- Génération automatique des relevés de notes
- Vérification de cohérence des données (pas de doublons)
- Alertes automatiques pour les absences

4.3 Exigences de qualité

- Fiabilité des calculs de notes
- Traçabilité des modifications
- Interface accessible et compréhensible

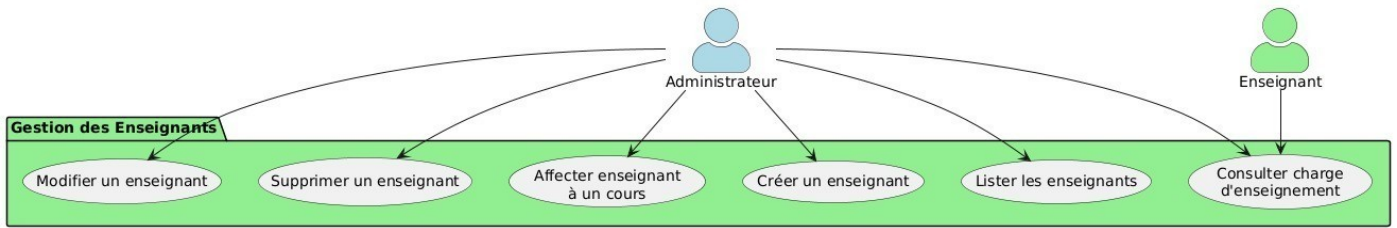
II-DIAGRAMMES DES CAS D'UTILISATION

Subdivisés en sept modules, notre diagramme des cas d'utilisation se présente lisible, mieux organisé et claire, facilitant la compréhension. L'outil utilisé est le **PlantUML**.



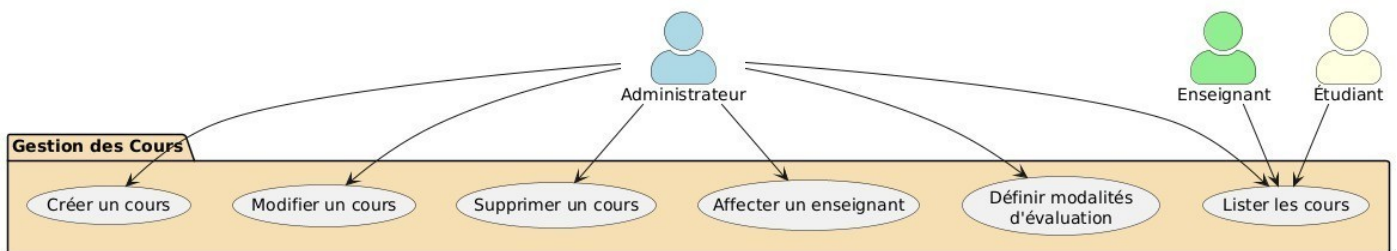
3-1 Gestion des Etudiants

MODULE 2: Gestion des Enseignants



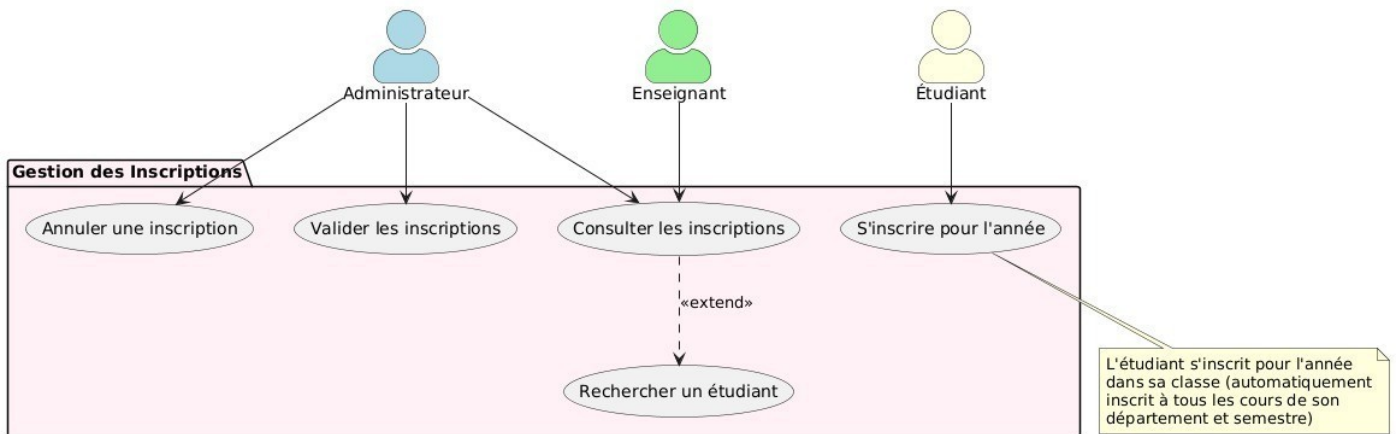
3-2 Gestion des Enseignants

MODULE 3: Gestion des Cours

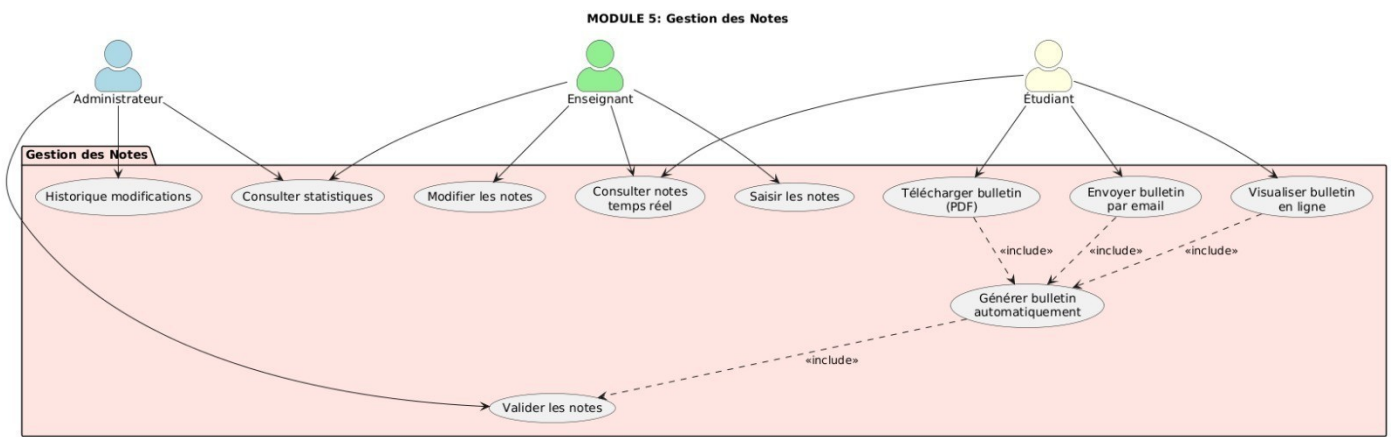


3-3 Gestion des cours

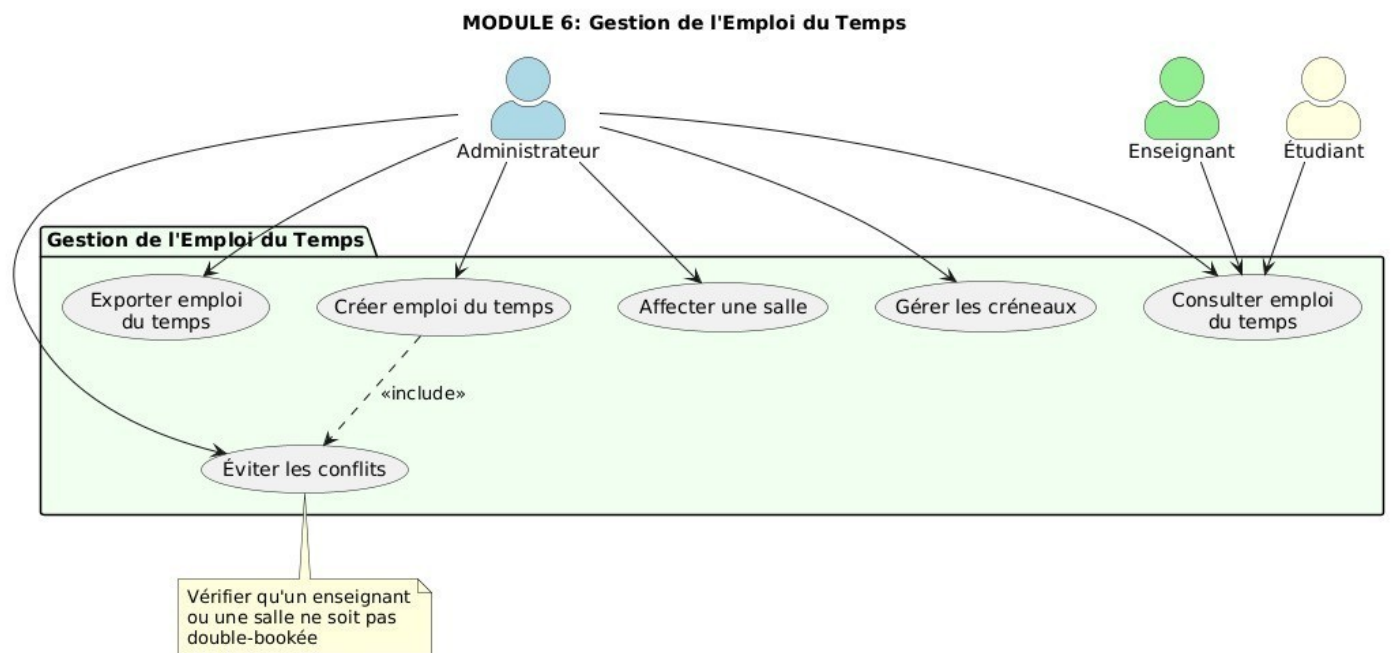
MODULE 4: Gestion des Inscriptions



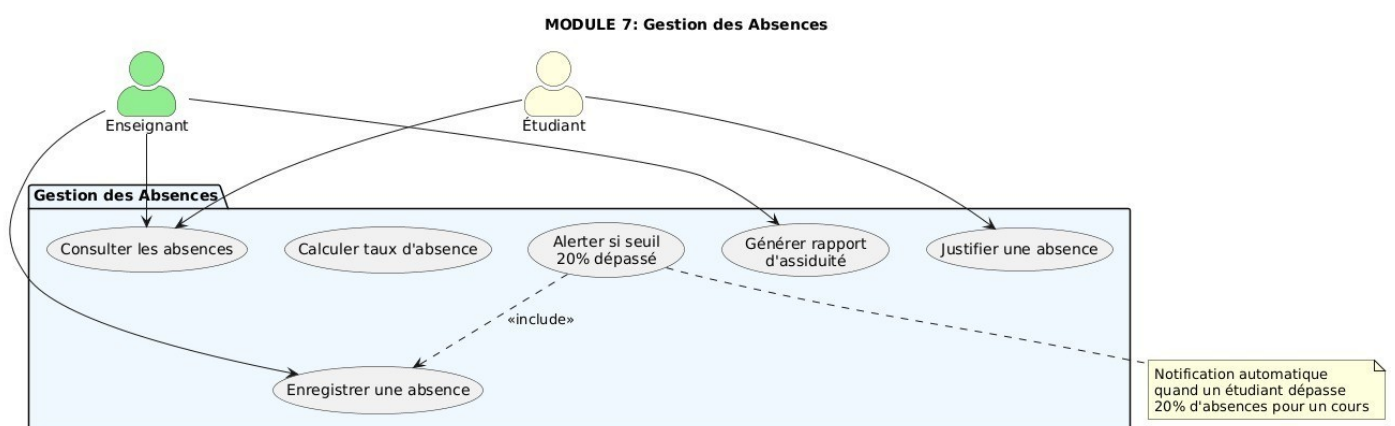
3-4 Gestion des Inscriptions



3-5 Gestion des Notes



3-6 Gestion de l'emploi du Temps



3-7 Gestion des Absences

III- REPRÉSENTATION TEXTUELLE DES CAS D'UTILISATION

CAS D'UTILISATION 1 : CRÉER UN COMPTE ÉTUDIANT

Identifiant : CU-01

Titre : Créer un compte étudiant

Acteur principal : Étudiant (futur)

Acteurs secondaires : Administrateur (pour validation)

Préconditions :

- L'étudiant n'a pas encore de compte dans le système
- La période d'inscription est ouverte
- L'étudiant est admis en L3 (GE ou GIT) Scénario nominal :
- L'étudiant accède à la page d'inscription du système
- Le système affiche le formulaire d'inscription avec les champs : matricule, nom, prénom, date de naissance, département (GE/GIT), niveau (L3), email, téléphone
- L'étudiant remplit tous les champs obligatoires
- L'étudiant crée un mot de passe
- L'étudiant confirme le mot de passe
- L'étudiant clique sur "S'inscrire"
- Le système vérifie que tous les champs sont remplis
- Le système vérifie que le matricule n'existe pas déjà
- Le système vérifie que l'email n'existe pas déjà
- Le système vérifie que les deux mots de passe correspondent
- Le système enregistre le compte avec le statut "En attente de validation"
- Le système affiche un message : "Compte créé avec succès. En attente de validation administrative."
- Le système envoie une notification à l'administrateur

Scénarios alternatifs :

A1 - Champ obligatoire manquant :

- À l'étape 9 : Un ou plusieurs champs obligatoires sont vides
- Le système affiche un message d'erreur : "Veuillez remplir tous les champs obligatoires"
- Retour à l'étape 3

A2 - Matricule déjà existant :

- À l'étape 10 : Le matricule existe déjà dans le système
- Le système affiche un message d'erreur : "Ce matricule existe déjà. Contactez l'administration si vous avez oublié votre mot de passe."
- Retour à l'étape 3
 - A3 - Email déjà utilisé :
- À l'étape 11 : L'email est déjà associé à un autre compte
- Le système affiche un message d'erreur : "Cet email est déjà utilisé"
- Retour à l'étape 3
 - A4 - Mots de passe non identiques :
- À l'étape 12 : Les deux mots de passe ne correspondent pas
- Le système affiche un message d'erreur : "Les mots de passe ne correspondent pas"
- Retour à l'étape 6
 - A5 - Période d'inscription fermée :
- À l'étape 1 : La période d'inscription est fermée
- Le système affiche un message : "La période d'inscription est fermée. Contactez l'administration."
- Fin du cas d'utilisation
 - A6 - Annulation :
- À tout moment avant l'étape 8 : L'étudiant clique sur "Annuler"
- Le système retourne à la page d'accueil
- Fin du cas d'utilisation

Postconditions :

- Le compte étudiant est créé avec le statut "En attente"
- Une notification est envoyée à l'administrateur
- L'étudiant ne peut pas encore se connecter (en attente de validation)
- L'administrateur peut valider ou rejeter l'inscription

CAS D'UTILISATION 2 : SAISIR LES NOTES

Identifiant : CU-02

Titre : Saisir les notes

Acteur principal : Enseignant

Acteurs secondaires : Système (calcul automatique)

Préconditions :

- L'enseignant est authentifié dans le système
- L'enseignant est affecté à au moins un cours
- Des étudiants de la classe L3 sont inscrits pour l'année
- Les modalités d'évaluation sont définies (pondérations CC, TP, Examen)

Scénario nominal :

1. L'enseignant se connecte au système
2. Le système affiche le tableau de bord enseignant
3. L'enseignant sélectionne "Saisir les notes"
4. Le système affiche la liste des cours assignés à l'enseignant
5. L'enseignant sélectionne un cours
6. Le système affiche la liste des étudiants inscrits au cours (matricule, nom, prénom)
7. Le système affiche un tableau de saisie avec colonnes : CC, TP, Examen, Moyenne
8. L'enseignant saisit la note de CC pour un étudiant (sur 20)
9. Le système vérifie que la note est entre 0 et 20
10. L'enseignant saisit la note de TP (sur 20)
11. Le système vérifie que la note est entre 0 et 20
12. L'enseignant saisit la note d'Examen (sur 20)
13. Le système vérifie que la note est entre 0 et 20
14. Le système calcule automatiquement la moyenne : $Moyenne = (CC \times 0.30 + TP \times 0.20 + Examen \times 0.50)$
15. Le système affiche la moyenne calculée et la mention (Très Bien ≥ 16 , Bien ≥ 14 , Assez Bien ≥ 12 , Passable ≥ 10 , Ajourné < 10)
16. L'enseignant répète les étapes 8-15 pour tous les étudiants
17. L'enseignant clique sur "Enregistrer"
18. Le système enregistre toutes les notes avec le statut "Non validée"
19. Le système affiche un message de confirmation : "Notes enregistrées avec succès"
20. Les étudiants peuvent consulter leurs notes immédiatement (consultation temps réel)

Scénarios alternatifs :

A1 - Note invalide :

- À l'étape 9, 11 ou 13 : Si la note saisie n'est pas entre 0 et 20
- Le système affiche un message d'erreur : "La note doit être comprise entre 0 et 20"

- Le système ne calcule pas la moyenne
- Retour à l'étape 8
- A2 - Note manquante :
- À l'étape 17 : Si toutes les notes (CC, TP, Examen) ne sont pas saisies pour un étudiant
- Le système affiche un avertissement : "Notes incomplètes pour certains étudiants"
- L'enseignant peut choisir de sauvegarder quand même (notes partielles)
- Le système n'affiche pas de moyenne pour les étudiants avec notes incomplètes
- A3 - Aucun étudiant inscrit :
- À l'étape 6 : Aucun étudiant de la classe n'est inscrit
- Le système affiche un message : "Aucun étudiant dans cette classe"
- Fin du cas d'utilisation A4 - Annulation :
- À tout moment avant l'étape 17 : L'enseignant clique sur "Annuler"
- Le système affiche une confirmation : "Voulez-vous vraiment annuler ? Les modifications ne seront pas sauvegardées"
- Si confirmation : retour au tableau de bord sans enregistrement
- Fin du cas d'utilisation

Postconditions :

- Les notes sont enregistrées dans le système
- Les moyennes sont calculées et affichées
- Les mentions sont attribuées
- Les étudiants peuvent consulter leurs notes immédiatement
- Les notes sont modifiables par l'enseignant (avant validation administrative)

CAS D'UTILISATION 3 : VALIDER LES NOTES

Identifiant : CU-03

Titre : Valider les notes

Acteur principal : Administrateur

Acteurs secondaires : Système (génération bulletin)

Préconditions :

- L'administrateur est authentifié
- Des notes ont été saisies par les enseignants

- Les notes sont au statut "Non validée" Scénario nominal :
 1. L'administrateur se connecte au système
 2. Le système affiche le tableau de bord administrateur
 3. L'administrateur sélectionne "Valider les notes"
 4. Le système affiche la liste des cours avec notes en attente de validation
 5. L'administrateur sélectionne un cours
 6. Le système affiche la liste des étudiants avec leurs notes (CC, TP, Examen, Moyenne)
 7. L'administrateur vérifie la cohérence des notes
 8. L'administrateur peut modifier une note si elle est incorrecte
 9. Le système recalcule automatiquement la moyenne si modification
 10. L'administrateur clique sur "Valider les notes du cours"
 11. Le système change le statut des notes à "Validée"
 12. Le système vérifie si TOUTES les notes du semestre sont validées
 13. Si toutes les notes du semestre sont validées, le système génère automatiquement le bulletin pour tous les étudiants
 14. Le système affiche un message de confirmation : "Notes validées avec succès"
 15. Si génération de bulletins : Le système affiche "Bulletins générés automatiquement pour tous les étudiants"
 16. Le système envoie une notification aux étudiants concernés

Scénarios alternatifs :

A1 - Notes incomplètes :

- À l'étape 7 : Si des notes sont manquantes (CC, TP ou Examen non saisis)
- Le système affiche un avertissement : "Attention : Notes incomplètes pour X étudiants"
- L'administrateur peut demander à l'enseignant de compléter
- L'administrateur peut annuler la validation
- A2 - Note incorrecte détectée :
 - À l'étape 7 : L'administrateur détecte une erreur
 - À l'étape 8 : L'administrateur modifie la note
 - À l'étape 9 : Le système recalcule la moyenne
 - Le système enregistre la modification dans l'historique
 - Retour à l'étape 10

A3 - Annulation de la validation :

- À tout moment avant l'étape 10 : L'administrateur clique sur "Annuler"
- Le système retourne au tableau de bord sans valider

- Fin du cas d'utilisation
A4 - Bulletins partiels :
- À l'étape 12 : Si toutes les notes du semestre ne sont pas encore validées
- Le système n'affiche que : "Notes validées avec succès pour ce cours"
- Les bulletins ne sont pas générés
- Les étudiants doivent attendre la validation de tous les cours

Postconditions :

- Les notes du cours sont validées
- Les notes ne sont plus modifiables par l'enseignant
- Si toutes les notes du semestre sont validées : bulletins générés automatiquement pour tous les étudiants de la classe
- Les étudiants reçoivent une notification
- L'historique des modifications est enregistré

CAS D'UTILISATION 4 : CRÉER EMPLOI DU TEMPS

Identifiant : CU-04

Titre : Créer emploi du temps

Acteur principal : Administrateur

Acteurs secondaires : Système (vérification conflits)

Préconditions :

- L'administrateur est authentifié
 - Des cours sont définis dans le système
 - Des enseignants sont affectés aux cours
 - Des salles sont disponibles dans le système
- Scénario nominal :
1. L'administrateur se connecte au système
 2. Le système affiche le tableau de bord administrateur
 3. L'administrateur sélectionne "Créer emploi du temps"
 4. Le système affiche le formulaire de création avec : semestre, département
 5. L'administrateur sélectionne le semestre (S1 ou S2) et le département (GE ou GIT)
 6. Le système affiche la grille horaire vide (Lundi à Vendredi, 8h00-18h00, pause 12h30-14h00)

7. Le système affiche la liste des cours à planifier pour ce département/semestre
8. L'administrateur sélectionne un cours dans la liste
9. Le système affiche les informations du cours (intitulé, enseignant, nombre d'heures)
10. L'administrateur sélectionne un créneau horaire (jour, heure début, heure fin)
11. L'administrateur sélectionne une salle
12. Le système vérifie qu'il n'y a pas de conflit :
 - o L'enseignant n'a pas déjà un cours à ce créneau
 - o La salle n'est pas déjà réservée à ce créneau
 - o Le créneau n'est pas pendant la pause déjeuner (12h30-14h00)
13. Si aucun conflit : le système affiche le cours dans la grille
14. L'administrateur répète les étapes 8-13 pour tous les cours
15. L'administrateur clique sur "Enregistrer l'emploi du temps"
16. Le système enregistre l'emploi du temps
17. Le système affiche un message de confirmation : "Emploi du temps créé avec succès"

Scénarios alternatifs :

A1 - Conflit enseignant :

- À l'étape 12 : L'enseignant a déjà un cours au même créneau
- Le système affiche un message d'erreur : "Conflit : [Nom enseignant] a déjà le cours [Nom cours] à ce créneau"
- Le système ne place pas le cours dans la grille
- Retour à l'étape 10 pour choisir un autre créneau
- À l'étape 12 : La salle est déjà réservée
- Le système affiche un message d'erreur : "Conflit : Salle [Numéro] déjà occupée par [Nom cours] à ce créneau"
- Le système ne place pas le cours
- L'administrateur peut choisir une autre salle ou un autre créneau
- Retour à l'étape 10 ou 11

A3 - Créneau pendant la pause :

- À l'étape 12 : Le créneau choisi chevauche 12h30-14h00
- Le système affiche un message : "Impossible : Pause déjeuner (12h30-14h00)" □
- Retour à l'étape 10

A4 - Capacité salle insuffisante :

- À l'étape 11 : La salle sélectionnée a une capacité inférieure au nombre d'étudiants
- Le système affiche un avertissement : "Attention : Capacité salle insuffisante (X places pour Y étudiants)"
- L'administrateur peut confirmer quand même ou choisir une autre salle
- A5 - Annulation :
- À tout moment avant l'étape 15 : L'administrateur clique sur "Annuler"
- Le système affiche une confirmation : "Voulez-vous vraiment annuler ?"
- Si confirmation : retour au tableau de bord sans enregistrement
- Fin du cas d'utilisation

Postconditions :

- L'emploi du temps est créé et enregistré
- Aucun conflit horaire n'existe
- Les enseignants et étudiants peuvent consulter l'emploi du temps
- L'emploi du temps peut être exporté en PDF

CAS D'UTILISATION 5 : ENREGISTRER UNE ABSENCE

Identifiant : CU-05

Titre : Enregistrer une absence

Acteur principal : Enseignant

Acteurs secondaires : Système (calcul taux, alerte)

Préconditions :

- L'enseignant est authentifié
 - L'enseignant a un cours planifié dans l'emploi du temps
 - Des étudiants de la classe sont inscrits pour l'année
 - La séance de cours est en cours ou passée
- Scénario nominal :
1. L'enseignant se connecte au système
 2. Le système affiche le tableau de bord enseignant
 3. L'enseignant sélectionne "Enregistrer les absences"
 4. Le système affiche la liste des cours de l'enseignant
 5. L'enseignant sélectionne un cours
 6. Le système affiche la date du jour et demande confirmation
 7. L'enseignant confirme la date (ou sélectionne une autre date)

8. Le système affiche la liste de tous les étudiants de la classe (matricule, nom, prénom)
9. Le système affiche des cases à cocher "Présent" / "Absent" pour chaque étudiant (par défaut "Présent")
10. L'enseignant coche "Absent" pour les étudiants absents
11. L'enseignant clique sur "Enregistrer"
12. Pour chaque absence, le système calcule le taux d'absence de l'étudiant pour ce cours
13. Le système vérifie si le taux d'absence dépasse 20%
14. Si taux > 20% : le système génère une alerte automatique
15. Le système affiche le message : "Absences enregistrées. X alerte(s) générée(s)"
16. Le système envoie une notification aux étudiants absents
17. Si alerte : le système notifie également l'étudiant du dépassement du seuil

Scénarios alternatifs :

A1 - Aucune absence :

- À l'étape 10 : Tous les étudiants sont présents
- L'enseignant clique sur "Enregistrer"
- Le système enregistre la présence de tous
- Message : "Aucune absence enregistrée pour cette séance"
- Fin du cas d'utilisation

A2 - Absence déjà enregistrée :

- À l'étape 6 : Des absences sont déjà enregistrées pour cette date
- Le système affiche un message : "Des absences sont déjà enregistrées pour cette séance. Voulez-vous les modifier ?"
- Si oui : le système affiche les absences existantes (cases pré-cochées)
- L'enseignant peut modifier
- Continuer à l'étape 10

A3 - Date future :

- À l'étape 7 : L'enseignant sélectionne une date future
- Le système affiche un avertissement : "Attention : Date future. Confirmezvous ?"
- Si confirmation : continuer
- Sinon : retour à l'étape 6

A4 - Seuil 20% dépassé :

- À l'étape 13-14 : Un ou plusieurs étudiants dépassent 20% d'absences
- Le système génère une alerte

- Le système affiche : "ALERTE : [Nom étudiant] a dépassé 20% d'absences (X% actuellement)"
- L'étudiant reçoit une notification automatique
- Les absences sont quand même enregistrées
- A5 - Annulation :
- À tout moment avant l'étape 11 : L'enseignant clique sur "Annuler"
- Retour au tableau de bord sans enregistrement
- Fin du cas d'utilisation

Postconditions :

- Les absences sont enregistrées pour la séance
- Le taux d'absence est mis à jour pour chaque étudiant absent
- Des alertes sont générées si taux > 20%
- Les étudiants absents sont notifiés
- L'enseignant peut générer un rapport d'assiduité

CAS D'UTILISATION 6 : CONSULTER NOTES EN TEMPS RÉEL

Identifiant : CU-06

Titre : Consulter notes en temps réel

Acteur principal : Étudiant

Acteurs secondaires : Aucun

Préconditions :

- L'étudiant est authentifié
 - L'étudiant a un compte validé
 - Au moins une note a été saisie par un enseignant
- Scénario nominal :
1. L'étudiant se connecte au système
 2. Le système affiche le tableau de bord étudiant
 3. L'étudiant sélectionne "Consulter mes notes"
 4. Le système affiche la liste de tous les cours de l'étudiant
 5. Pour chaque cours, le système affiche :
 - Code et intitulé du cours
 - Note CC (si saisie)
 - Note TP (si saisie)

- o Note Examen (si saisie)
 - o Moyenne du cours (si toutes les notes sont saisies)
 - o Mention (si moyenne calculée)
 - o Statut : "Validée" ou "Non validée"
6. Le système affiche la moyenne générale (si au moins un cours a une moyenne)
 7. L'étudiant peut cliquer sur un cours pour voir les détails
 8. Le système affiche les détails : date de saisie, enseignant, statistiques du cours (moyenne de classe)

Scénarios alternatifs :

A1 - Aucune note saisie :

- À l'étape 4 : Aucune note n'a été saisie
- Le système affiche : "Aucune note disponible pour le moment"
- Fin du cas d'utilisation

A2 - Notes partielles :

- À l'étape 5 : Certaines notes sont manquantes (ex : CC et TP saisis, mais pas Examen)
- Le système affiche les notes disponibles
- Le système affiche "En attente" pour les notes manquantes
- Aucune moyenne n'est affichée tant que toutes les notes ne sont pas saisies

A3 - Nouvelle note ajoutée :

- Pendant la consultation : Un enseignant saisit une nouvelle note
- L'étudiant clique sur "Actualiser"
- Le système recharge les données
- La nouvelle note apparaît immédiatement (temps réel)

Postconditions :

- L'étudiant a consulté ses notes
- Aucune modification n'est effectuée dans le système

IV- IDENTIFICATION DES CLASSES D'OBJETS

1. CLASSE UTILISATEUR (ABSTRAITE)

Description : Classe abstraite représentant tout utilisateur du système

Attributs :

- idUtilisateur : int - Identifiant unique
- nom : string - Nom de famille
- prenom : string - Prénom
- email : string - Adresse email
- telephone : string - Numéro de téléphone
- login : string - Identifiant de connexion
- motDePasse : string - Mot de passe crypté
- statut : string - Statut du compte (Actif, Inactif, En attente)
- dateCreation : date - Date de création du compte

Opérations :

- authentifier(login, motDePasse) : Utilisateur* - Authentifier un utilisateur et retourner son instance (Etudiant, Enseignant ou Administrateur) selon son rôle dans la base de données
- seDeconnecter() : void - Fermer la session
- modifierProfil(nom, prenom, email, telephone) : boolean - Mettre à jour ses informations
- modifierMotDePasse(ancienMdp, nouveauMdp) : boolean - Changer son mot de passe

2. CLASSE ADMINISTRATEUR

Description : Gestionnaire du système avec tous les droits (hérite de Utilisateur)

Attributs :

- (hérite tous les attributs de Utilisateur)
- fonction : string - Fonction administrative

Opérations :

Gestion des étudiants :

- `creerEtudiant(données)` : Etudiant - Créer un compte étudiant
- `modifierEtudiant(idEtudiant, données)` : boolean - Modifier informations étudiant
- `supprimerEtudiant(idEtudiant)` : boolean - Supprimer un étudiant
- `validerInscription(idEtudiant)` : boolean - Valider inscription d'un étudiant
- `rejeterInscription(idEtudiant, motif)` : boolean - Rejeter une inscription

Gestion des enseignants :

- `creerEnseignant(données)` : Enseignant - Créer un compte enseignant
- `modifierEnseignant(idEnseignant, données)` : boolean - Modifier informations enseignant
- `supprimerEnseignant(idEnseignant)` : boolean - Supprimer un enseignant
- `affecterCours(idEnseignant, idCours)` : boolean - Affecter un enseignant à un cours

Gestion des cours :

- `creerCours(données)` : Cours - Créer un nouveau cours
- `modifierCours(idCours, données)` : boolean - Modifier un cours
- `supprimerCours(idCours)` : boolean - Supprimer un cours

Gestion des notes :

- `validerNotes(idCours)` : boolean - Valider les notes d'un cours
- `modifierNote(idNote, nouvelleValeur)` : boolean - Corriger une note si erreur

Gestion emploi du temps :

- `creerEmploiDuTemps(semester, departement)` : EmploiDuTemps - Créer emploi du temps
- `ajouterSeance(idCours, jour, heureDebut, heureFin, salle)` : Seance - Ajouter un créneau

Statistiques et rapports :

- `consulterStatistiques()` : Rapport - Générer statistiques globales du système

3. CLASSE ENSEIGNANT

Description : Enseignant responsable de cours (hérite de Utilisateur)

Attributs :

- (hérite tous les attributs de Utilisateur)
- matricule : string - Matricule unique de l'enseignant
- specialite : string - Domaine de spécialité (Réseaux, Électronique, etc.)
- grade : string - Grade académique (Assistant, Maître-assistant, Professeur)
- departement : string - Département (GE ou GIT)

Opérations :

- consulterMesCours() : List<Cours> - Liste des cours assignés
- consulterEtudiants(idCours) : List<Etudiant> - Liste des étudiants d'un cours
- saisirNotes(idCours, listeNotes) : boolean - Saisir notes (CC, TP, Examen)
- modifierNotes(idNote, nouvelleValeur) : boolean - Modifier notes avant validation
- enregistrerAbsence(idCours, date, listeAbsents) : boolean - Enregistrer absences d'une séance
- consulterAbsences(idCours) : List<Absence> - Consulter absences d'un cours
- genererRapportAssiduite(idCours) : Rapport - Rapport d'assiduité par cours
- consulterEmploiDuTemps() : EmploiDuTemps - Consulter son emploi du temps personnel
- consulterStatistiquesCours(idCours) : Statistique - Statistiques d'un cours (moyenne classe, taux réussite)

4. CLASSE ETUDIANT

Description : Étudiant de niveau L3 (hérite de Utilisateur)

Attributs :

- (hérite tous les attributs de Utilisateur)
- matricule : string - Matricule unique de l'étudiant

- dateNaissance : date - Date de naissance
- departement : string - Département (GE ou GIT)
- niveau : string - Niveau d'études (L3)
- anneeInscription : int - Année académique d'inscription

Opérations :

- creerCompte(données) : boolean - Créer son compte (inscription administrative)
- getAll() : List<Etudiant> - Récupérer tous les étudiants depuis la base de données (méthode statique utilisée pour charger les listes d'étudiants)
- getByMatricule(matricule) : Etudiant - Récupérer un étudiant spécifique par son matricule (méthode statique)
- consulterNotes() : List<Note> - Consulter toutes ses notes
- consulterNotesCours(idCours) : Note - Consulter notes d'un cours spécifique
- calculerMoyenneGenerale() : float - Calculer sa moyenne générale du semestre
- consulterAbsences() : List<Absence> - Consulter toutes ses absences
consulterAbsencesCours(idCours) : List<Absence> - Absences d'un cours spécifique
- justifierAbsence(idAbsence, motif, justificatif) : boolean - Justifier une absence
- telechargerBulletin() : Bulletin - Télécharger son bulletin en PDF
- visualiserBulletin() : Bulletin - Visualiser bulletin en ligne
- recevoirNotification(message) : void - Recevoir notifications (nouvelles notes, bulletin disponible)
- consulterEmploiDuTemps() : EmploiDuTemps - Consulter son emploi du temps
- modifierContact(email, telephone) : boolean - Modifier email et téléphone

5. CLASSE COURS

Description : Cours enseigné dans un département

Attributs :

- idCours : int - Identifiant unique
- code : string - Code du cours (ex : INF301, ELEC202)
- intitule : string - Intitulé complet du cours
- creditsECTS : int - Nombre de crédits ECTS
- nombreHeures : int - Volume horaire total
- semestre : string - Semestre (S1 ou S2)
- departement : string - Département (GE ou GIT)
- pondérationCC : float - Pondération Contrôle Continu (0.30)
- pondérationTP : float - Pondération Travaux Pratiques (0.20)
- pondérationExamen : float - Pondération Examen (0.50)

Opérations :

- listerEtudiants() : List<Etudiant> - Liste des étudiants inscrits au cours
- calculerMoyenneCours(noteCC, noteTP, noteExamen) : float - Calculer moyenne selon pondérations
- obtenirStatistiques() : Statistique - Statistiques du cours (moyenne classe, taux réussite)
- getEnseignant() : Enseignant - Obtenir l'enseignant responsable

6. CLASSE INSCRIPTION

Description : Inscription d'un étudiant pour une année académique

Attributs :

- idInscription : int - Identifiant unique
- idEtudiant : int - Référence vers l'étudiant
- dateInscription : date - Date de demande d'inscription
- anneeAcademique : string - Année académique (ex : 2025-2026)
- statut : string - Statut (En attente, Validée, Rejetée)
- dateValidation : date - Date de validation par admin

Opérations :

- getStatut() : string - Obtenir le statut de l'inscription
- estValidee() : boolean - Vérifier si inscription validée

7. CLASSE NOTE

Description : Note d'un étudiant pour un cours

Attributs :

- idNote : int - Identifiant unique
- idEtudiant : int - Référence vers l'étudiant
- idCours : int - Référence vers le cours
- noteCC : float - Note de Contrôle Continu (sur 20)
- noteTP : float - Note de Travaux Pratiques (sur 20)
- noteExamen : float - Note d'Examen (sur 20)
- moyenne : float - Moyenne calculée automatiquement
- mention : string - Mention (Très Bien, Bien, Assez Bien, Passable, Ajourné)
- validee : boolean - Statut de validation
- dateSaisie : date - Date de saisie par l'enseignant
- dateValidation : date - Date de validation par l'admin

Opérations :

Opérations de consultation :

- getAll() : List<Note> - Récupérer toutes les notes (méthode statique)

- getByCourse(cours) : List<Note> - Récupérer les notes d'un cours spécifique
- getByMatricule(matricule) : List<Note> - Récupérer les notes d'un étudiant
- getNotesEnAttente() : List<Note> - Récupérer uniquement les notes avec statut "Soumise" en attente de validation administrative
- getCourseCodesWithStatus(statut) : List<string> - Récupérer la liste des codes de cours ayant des notes avec un statut donné
- getByStatusAndCourseCode(statut, code) : List<Note> - Filtrer les notes par statut et code de cours

Opérations de modification :

- upsert(note) : void - Insérer ou mettre à jour une note (insertion si inexistante, mise à jour sinon)
- updateValidation(matricule, cours, statut, commentaire) : boolean - Valider ou rejeter une note avec commentaire administrateur
- calculerMoyenne() : float - Calculer moyenne selon pondérations du cours
- attribuerMention() : string - Attribuer mention selon la moyenne
- estComplete() : boolean - Vérifier si toutes les notes sont saisies
- estValidee() : boolean - Vérifier si note validée par admin

Opérations utilitaires :

- nowStr() : string - Obtenir la date/heure actuelle au format ISO (utilisée pour les horodatages)
- courseCode(cours) : string - Extraire le code du cours depuis l'intitulé

8. CLASSE ABSENCE

Description : Absence d'un étudiant à une séance de cours

Attributs :

- idAbsence : int - Identifiant unique
- idEtudiant : int - Référence vers l'étudiant
- idCours : int - Référence vers le cours
- dateAbsence : date - Date de l'absence
- justifiee : boolean - Absence justifiée ou non
- motif : string - Motif de l'absence (si justifiée)
- justificatif : string - Document justificatif (fichier ou texte)

Opérations :

- `getByMatricule(matricule)` : `List<Absence>` - Récupérer toutes les absences d'un étudiant (méthode statique)
- `justifier(idAbsence, justification)` : `boolean` - Justifier une absence avec motif et justificatif
- `enregistrerAbsences(etudiants, indices, cours, date)` : `int` - Enregistrer plusieurs absences simultanément pour une séance de cours donnée. Retourne le nombre d'absences enregistrées
- `estJustifiee()` : `boolean` - Vérifier si absence justifiée -
`calculerTauxAbsence(nombreSeancesTotales)` : `float` - Calculer taux d'absence

9. CLASSE SEANCE

Description : Créneau horaire d'un cours dans l'emploi du temps (renommée de `EmploiDuTemps`)

Attributs :

- `idSeance` : `int` - Identifiant unique
- `idCours` : `int` - Référence vers le cours
- `jour` : `string` - Jour de la semaine (Lundi, Mardi, etc.)
- `heureDebut` : `time` - Heure de début
- `heureFin` : `time` - Heure de fin
- `idSalle` : `int` - Référence vers la salle
- `semestre` : `string` - Semestre concerné (S1 ou S2)

Opérations : • `verifierConflitEnseignant(idEnseignant, jour, heureDebut, heureFin)` : `boolean` - Vérifier conflit enseignant

- `verifierConflitSalle(idSalle, jour, heureDebut, heureFin)` : `boolean` - Vérifier conflit salle
- `estPendantPause()` : `boolean` - Vérifier si créneau pendant pause déjeuner (12h30-14h00)

10. CLASSE SALLE

Description : Salle de cours

Attributs :

- idSalle : int - Identifiant unique
- numeroSalle : string - Numéro de la salle (ex : A101, B205)
- capacite : int - Capacité d'accueil (nombre de places)
- type : string - Type de salle (Amphi, TD, TP, Labo)
- batiment : string - Bâtiment où se trouve la salle

Opérations :

- verifierDisponibilite(jour, heureDebut, heureFin) : boolean - Vérifier si salle disponible
- estCapaciteSuffisante(nombreEtudiants) : boolean - Vérifier si capacité suffisante

11. CLASSE BULLETIN

Description : Bulletin de notes d'un étudiant pour un semestre

Attributs :

- idBulletin : int - Identifiant unique
- idEtudiant : int - Référence vers l'étudiant
- semestre : string - Semestre concerné (S1 ou S2)
- anneeAcademique : string - Année académique
- moyenneGenerale : float - Moyenne générale du semestre
- mentionGlobale : string - Mention globale du semestre
- dateGeneration : date - Date de génération automatique
- valide : boolean - Bulletin validé (toutes notes validées)

Opérations :

- generer() : boolean - Générer le bulletin automatiquement
- telecharger() : file - Télécharger en format PDF
- envoyerParEmail(email) : boolean - Envoyer le bulletin par email
- visualiser() : Bulletin - Afficher le bulletin en ligne

- calculerMoyenneGenerale() : float - Calculer moyenne générale (pondérée par crédits)

12. CLASSE EMPLOIDUTEMPS

Description : Emploi du temps complet d'un semestre pour un département

Attributs :

- idEmploiDuTemps : int - Identifiant unique
- semestre : string - Semestre (S1 ou S2)
- anneeAcademique : string - Année académique
- departement : string - Département (GE ou GIT)
- dateCreation : date - Date de création
- valide : boolean - Emploi du temps validé

Opérations :

- ajouterSeance(seance) : boolean - Ajouter une séance
- supprimerSeance(idSeance) : boolean - Supprimer une séance
- verifierCoherence() : boolean - Vérifier absence de conflits
- exporter() : file - Exporter en PDF
- consulterParEnseignant(idEnseignant) : List<Seance> - Emploi du temps d'un enseignant
- consulterParEtudiant(departement, niveau) : List<Seance> - Emploi du temps d'une classe

V- DIAGRAMMES DE SEQUENCE

Nous avons considéré quatre cas d'utilisation importants à savoir: le CU-01, le CU-02, le CU-03 et le CU-04. L'outil utilisé est le **PlantUML**.

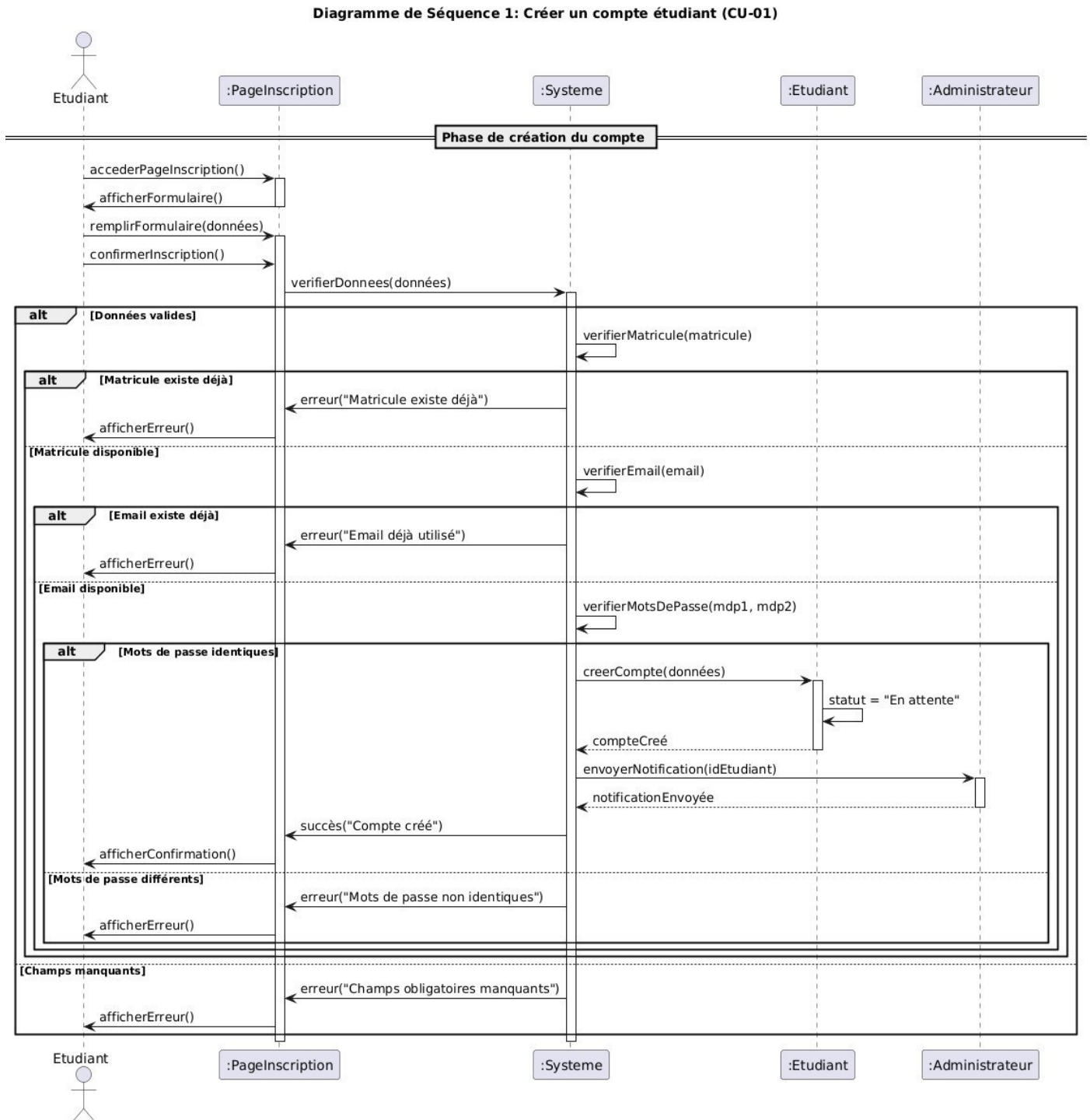


Fig 1: 5-1 CU01

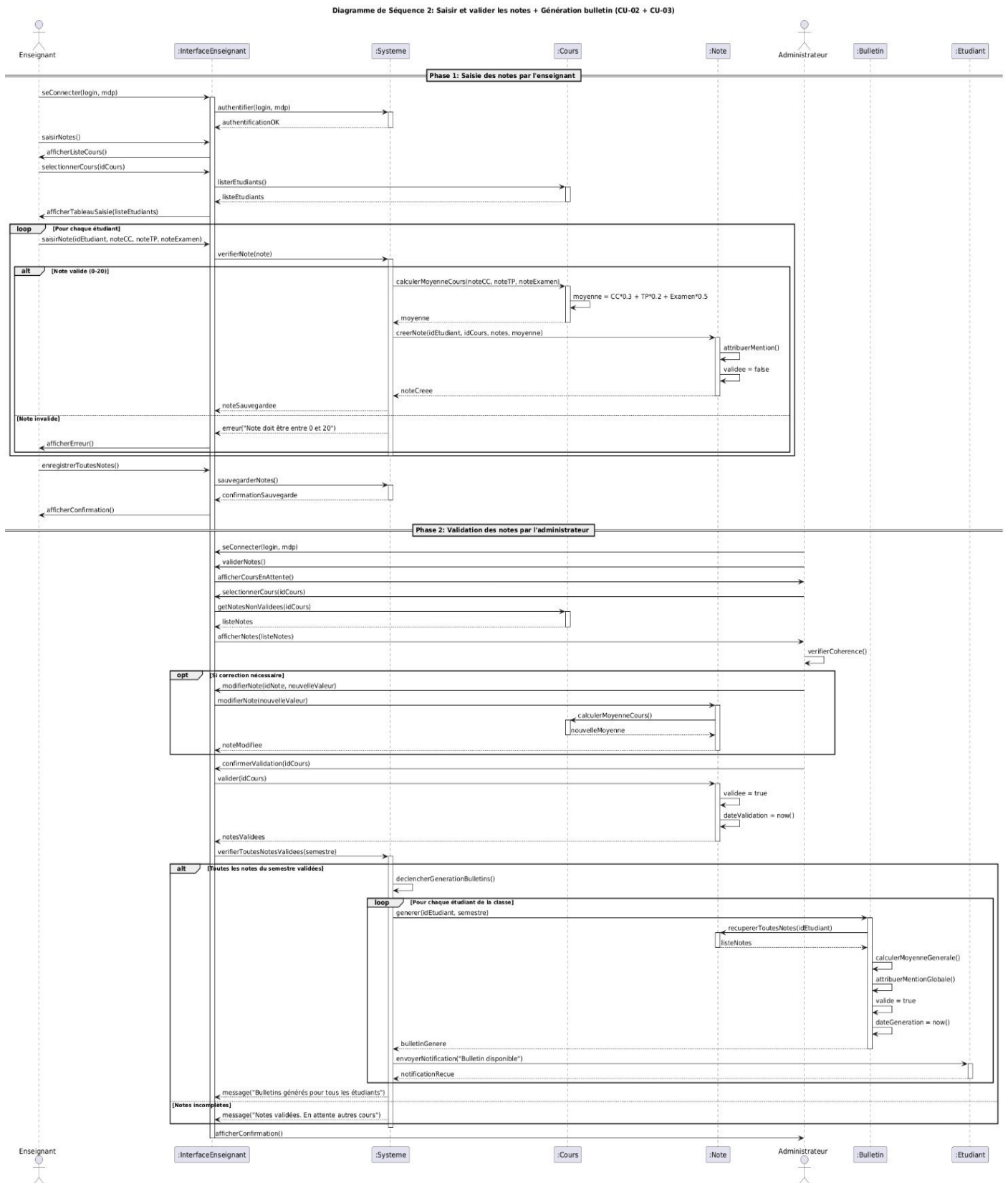


Fig2: 5-2 CU02+CU03

Diagramme de Séquence 3: Créer emploi du temps avec vérification conflits (CU-04)

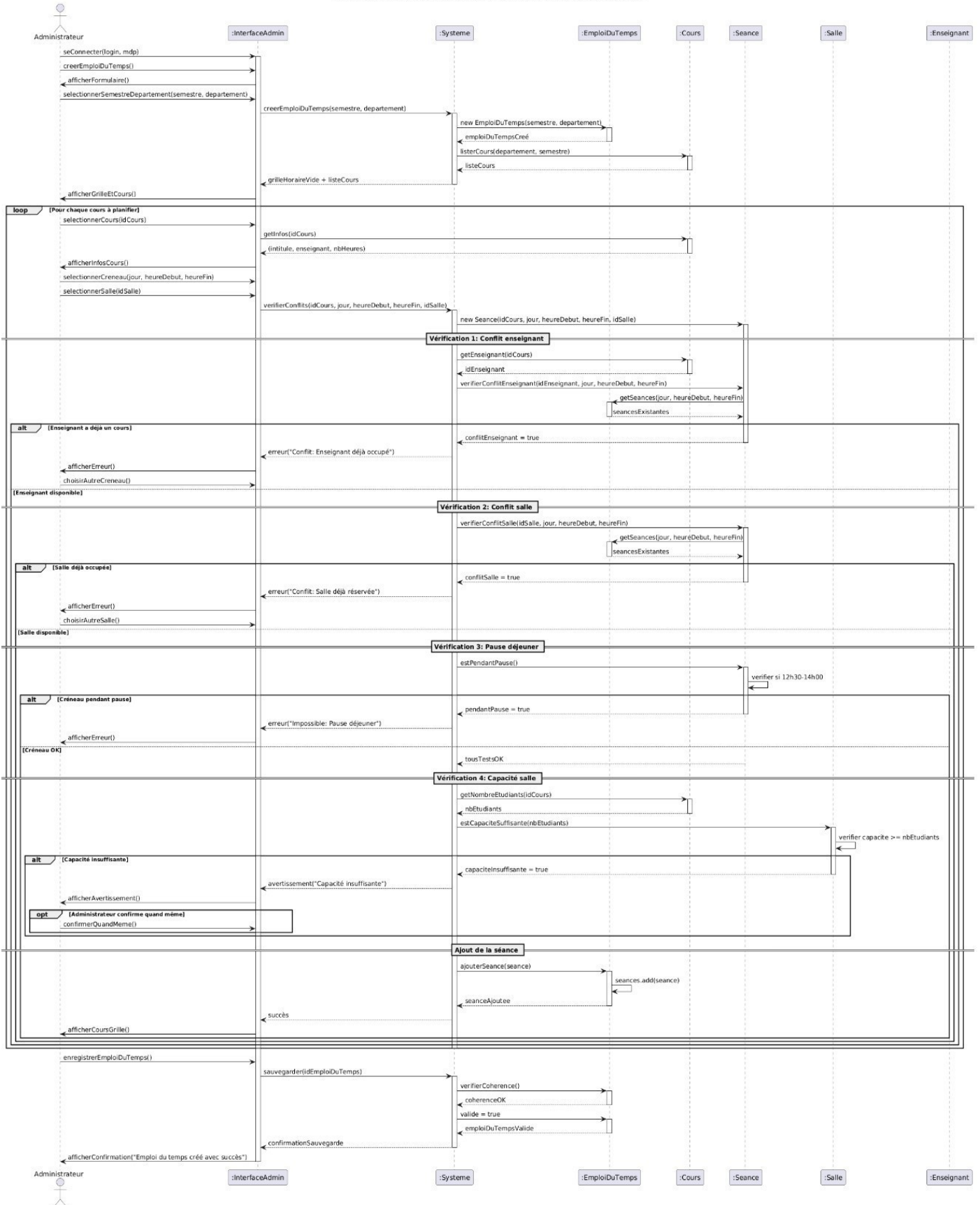


Diagramme de Séquence 3: Créer emploi du temps avec vérification conflits (CU-04)

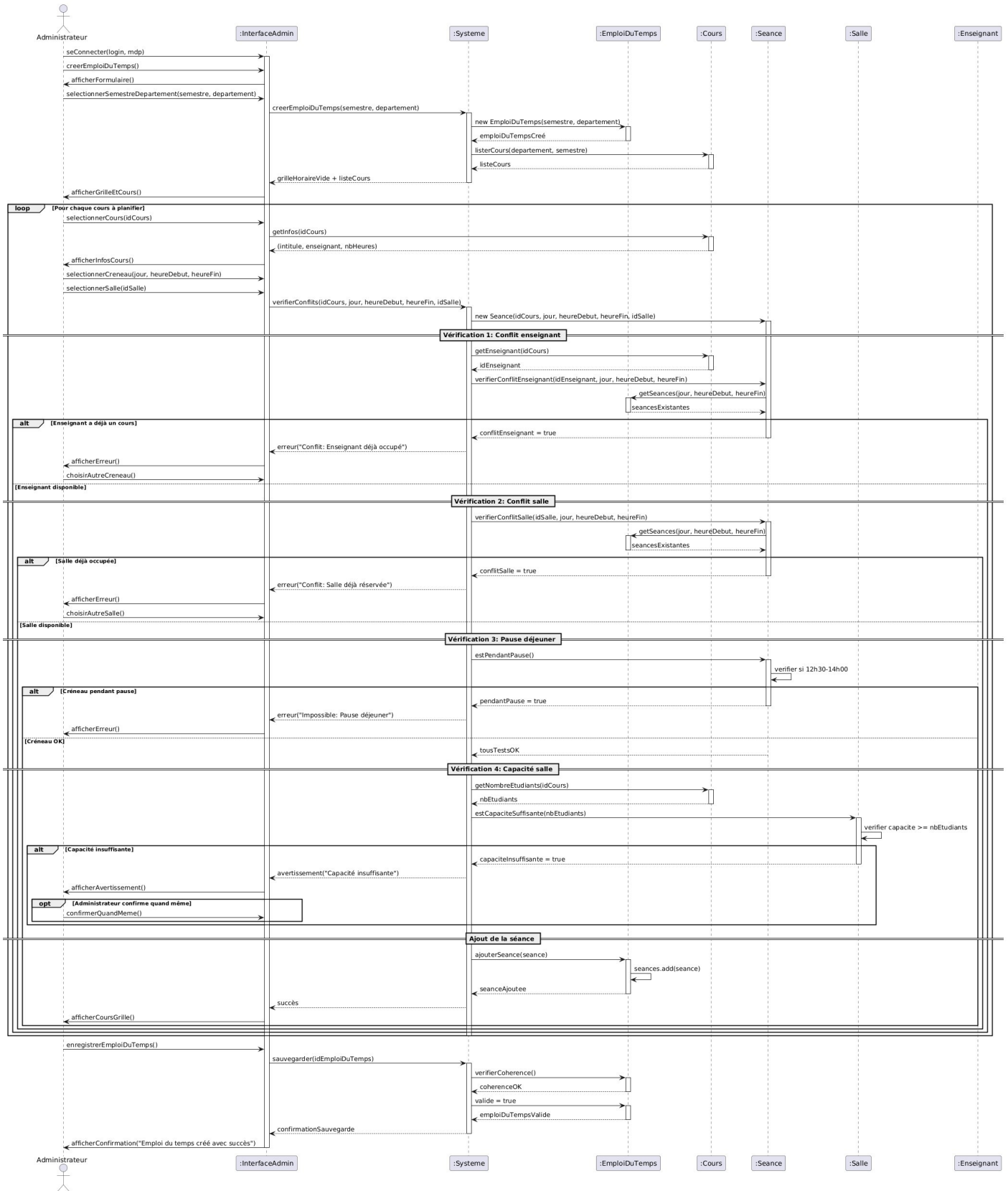


Fig3:5-3 CU04

VI- DIAGRAMME DE CLASSES

En conception UML, les acteurs du diagramme de cas d'utilisation sont intégrés au diagramme de classes lorsqu'ils possèdent des données ou des comportements que le système doit gérer. Concrètement, un acteur humain (comme un Étudiant ou un Enseignant) devient une classe à part entière pour stocker ses informations personnelles et définir ses droits d'accès. Ces classes sont souvent organisées par un lien d'héritage vers une classe mère (par exemple, Utilisateur) afin de centraliser les attributs communs.

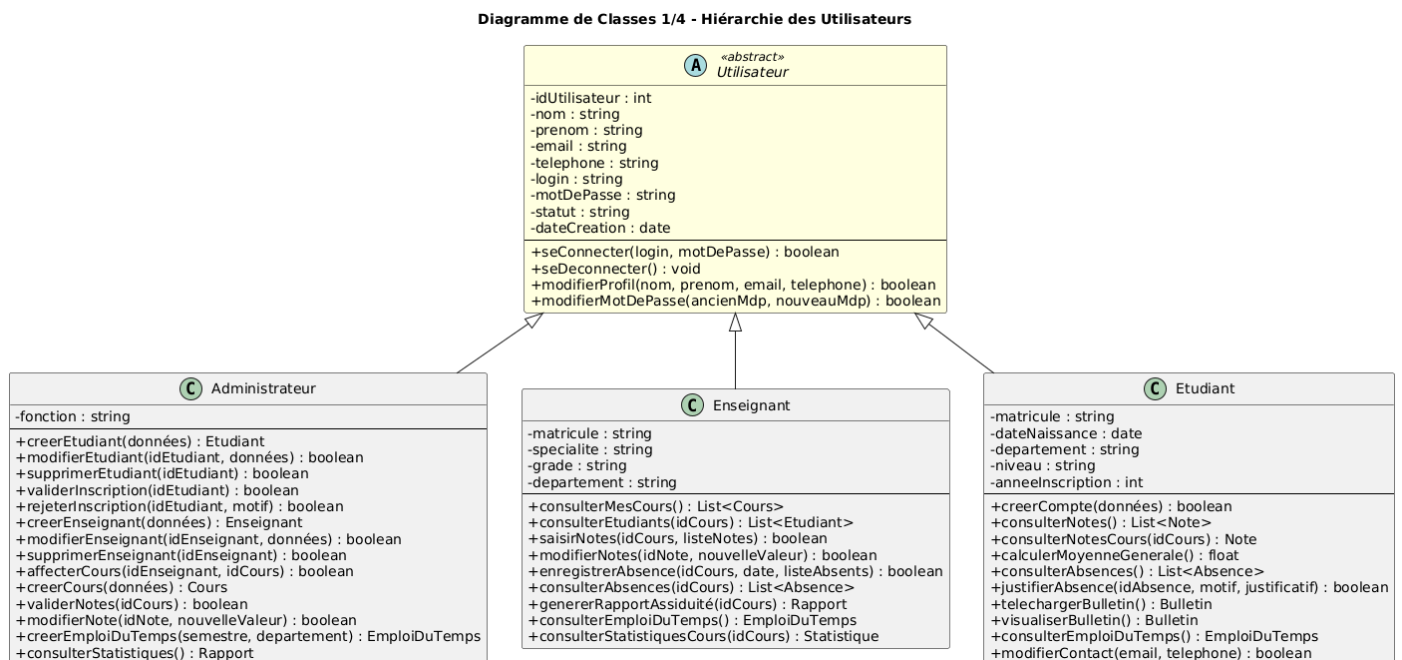


Fig4:DC1

Diagramme de Classes 2/4 - Gestion Académique (Cours, Notes, Bulletin)

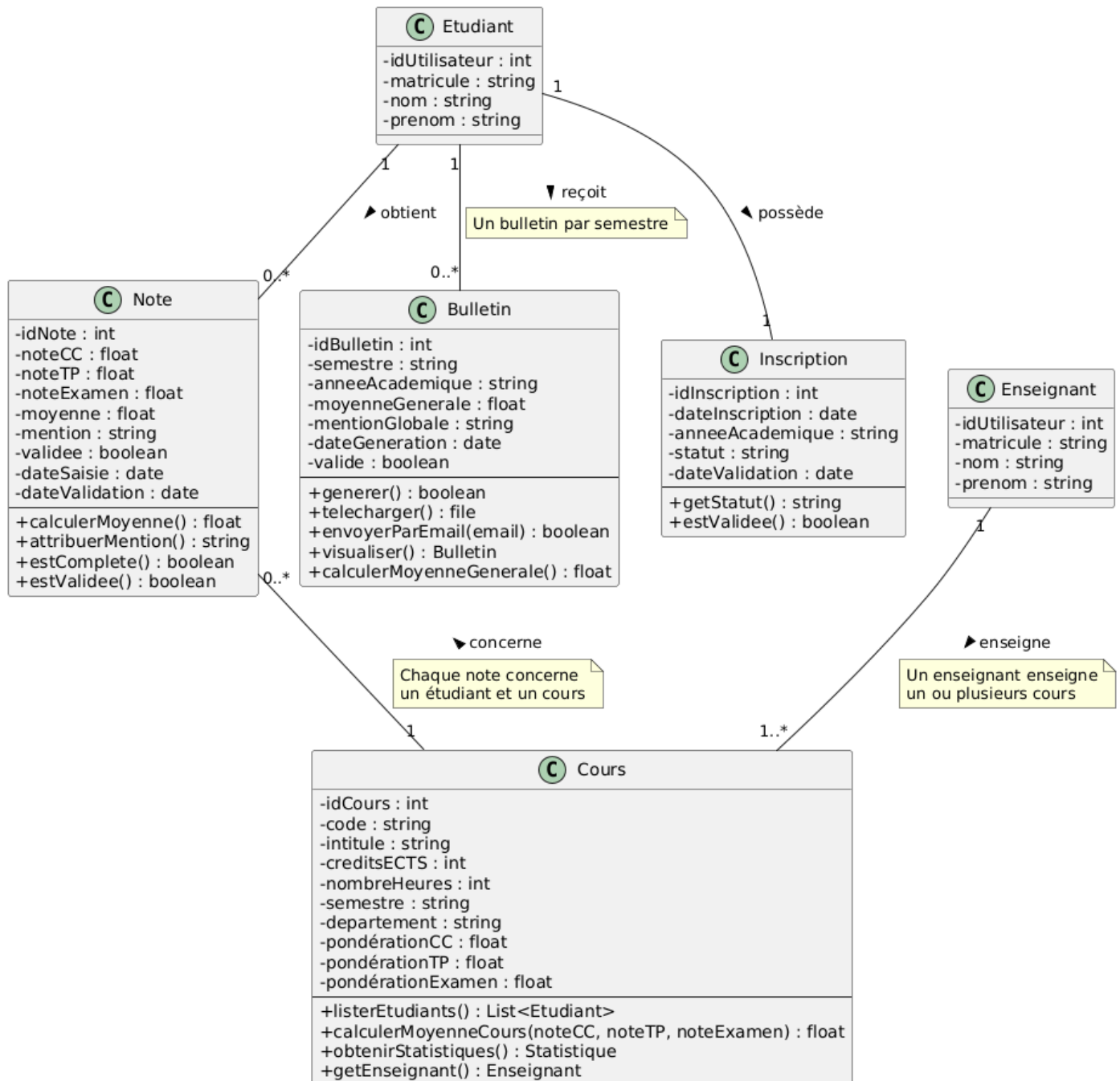


Fig5:DC2

Diagramme de Classes 3/4 - Gestion des Absences

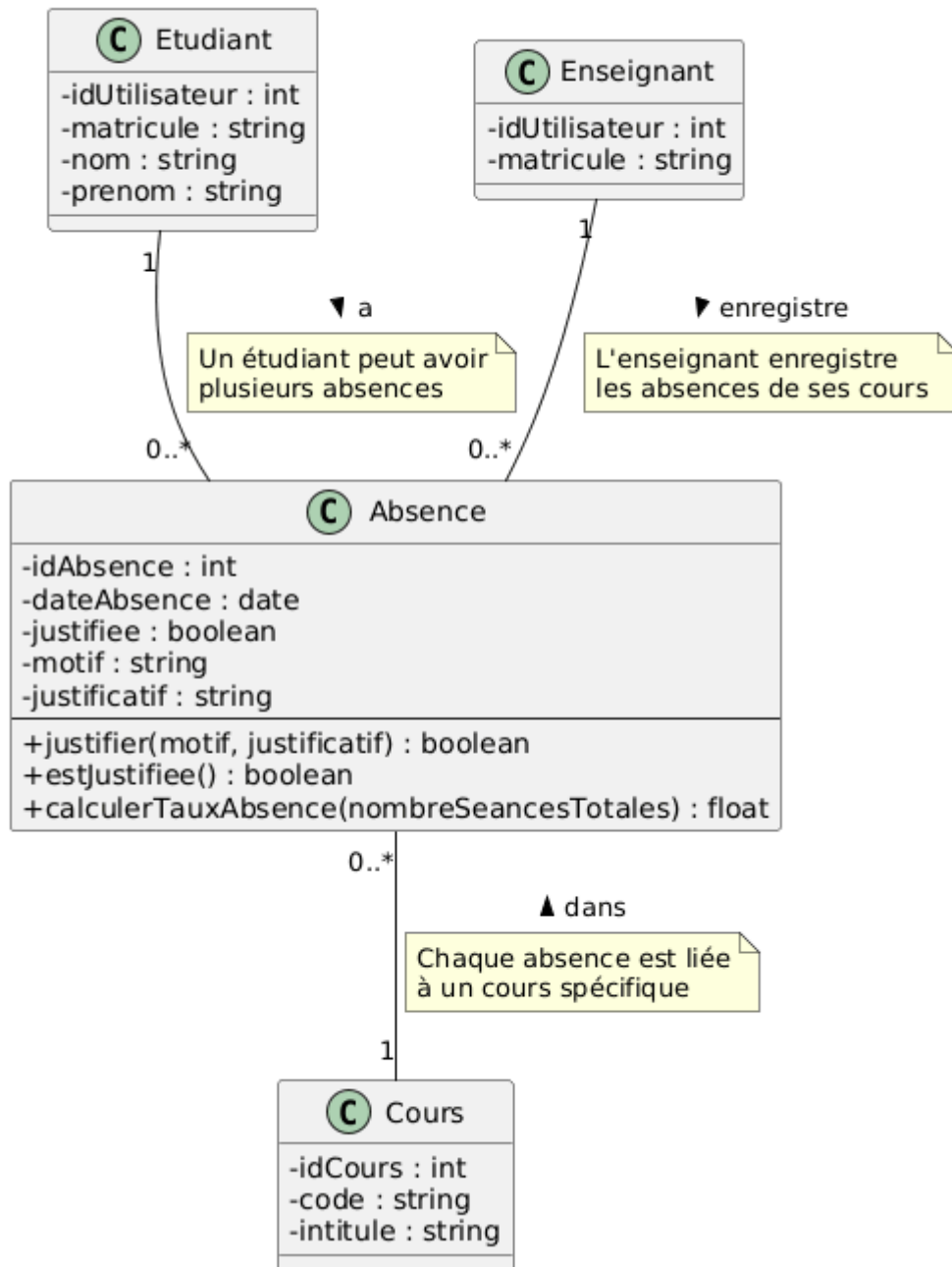


Fig6:DC3

Diagramme de Classes 4/4 - Gestion de l'Emploi du Temps

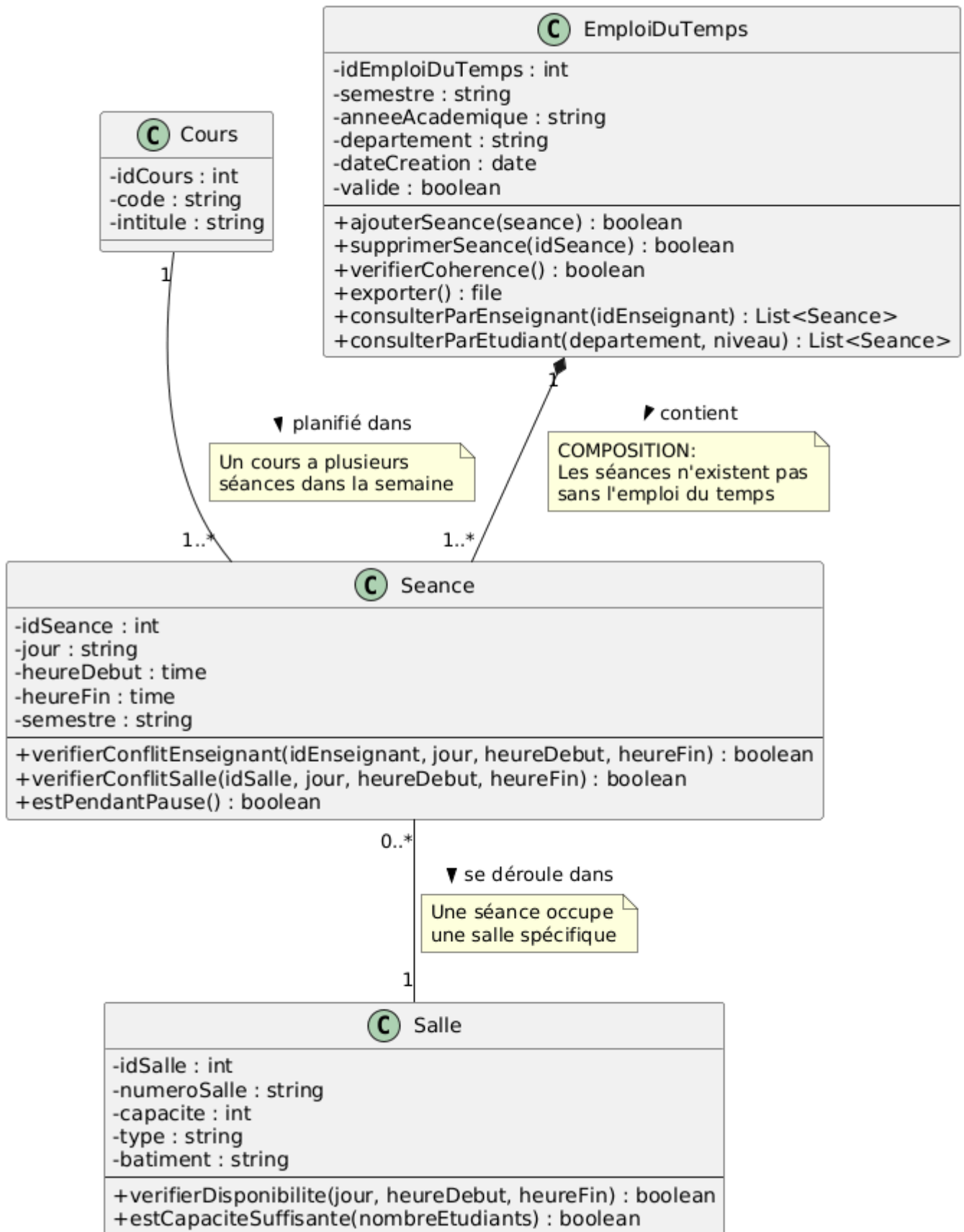


Fig7:DC4

Le diagramme de classes ci-dessous présente l'architecture réellement implémentée du système après refactoring, organisée en trois packages principaux avec séparation stricte des responsabilités selon le pattern MVC (Modèle-Vue-Contrôleur) :

- **Modèles métier (models/)** : Classes intelligentes avec méthodes statiques encapsulant la logique métier. Chaque modèle agit comme une façade d'accès aux données (pattern Active Record simplifié).
- **Accès aux données (database/)** : Classe Database avec méthodes statiques d'accès à SQLite3. Cette couche est accessible uniquement via les modèles, jamais directement depuis l'interface graphique.
- **Interface graphique - Fenêtres (frames/)** : Fenêtres principales de l'application (LoginFrame, AdminFrame, EnseignantFrame, EtudiantFrame). Responsables uniquement de l'affichage et de l'orchestration des interactions utilisateur.
- **Interface graphique - Dialogues (dialogs/)** : Boîtes de dialogue fonctionnelles (ValidationNotesDialog, SaisieNotesDialog, AbsencesDialog, ConsultationNotesDialog, ConsultationAbsencesDialog).

Relations architecturales clés :

- Les frames et dialogs dépendent des modèles (flèches pointillées ---->)
- Les modèles utilisent la Database pour la persistance (flèches pointillées ---->)
- Les frames ne connaissent pas l'existence de la Database (principe d'encapsulation)
- Utilisateur est une classe abstraite dont héritent Etudiant, Enseignant et Administrateur

Cette architecture garantit la maintenabilité, la testabilité et la portabilité du code.

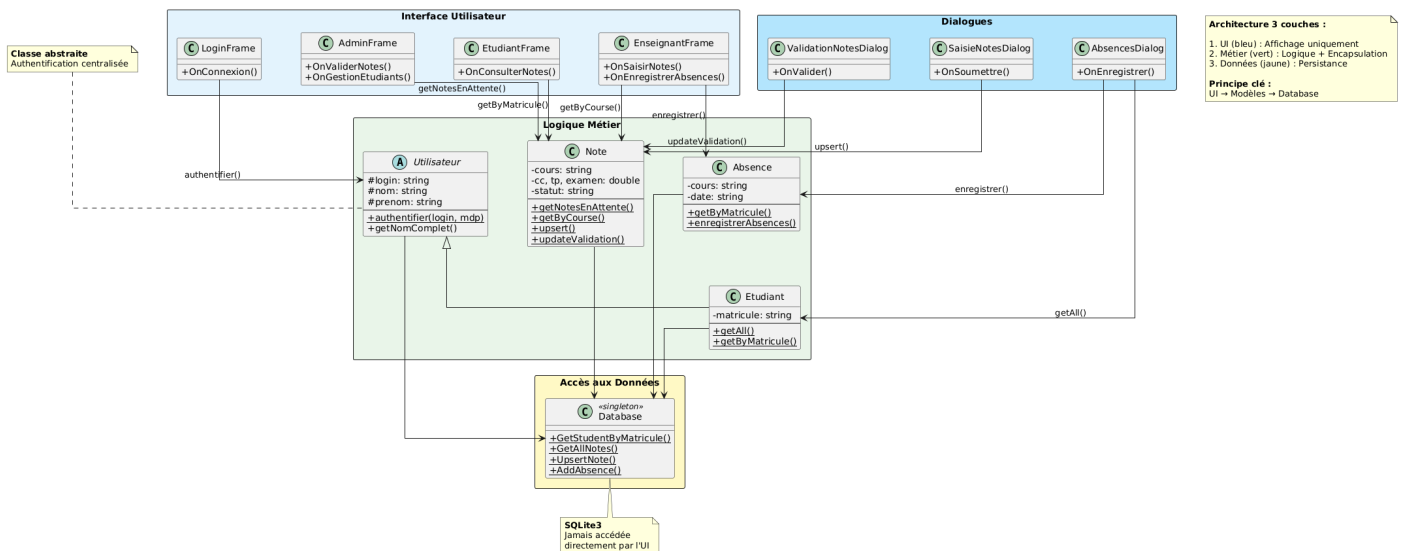


Fig8:- Diagramme de classes implémentation réelle

VII- CONCEPTION DES ECRANS AVEC WXWIDGETS

7.1 Introduction et choix technologiques

Pour cette étape cruciale, nous avons opté pour wxWidgets, une bibliothèque C++ multiplateforme reconnue pour sa robustesse et sa capacité à créer des interfaces natives sur différents systèmes d'exploitation (Windows, Linux, macOS). Cette bibliothèque offre une large gamme de composants (boutons, grilles, dialogues, menus) qui facilitent l'implémentation de nos cas d'utilisation.

7.2 État d'avancement de l'implémentation

À ce stade du projet, nous avons entamé la phase d'implémentation concrète de l'application. Les fondations du système sont en place et l'architecture logicielle est opérationnelle.

Composants actuellement implémentés:

1. Écran de connexion (LoginFrame)

L'écran d'authentification constitue le point d'entrée de l'application. Il permet:

- La sélection du rôle (Administrateur, Enseignant, Étudiant)
- La saisie sécurisée des identifiants avec option d'affichage du mot de passe
- La validation des champs et la redirection vers le tableau de bord approprié

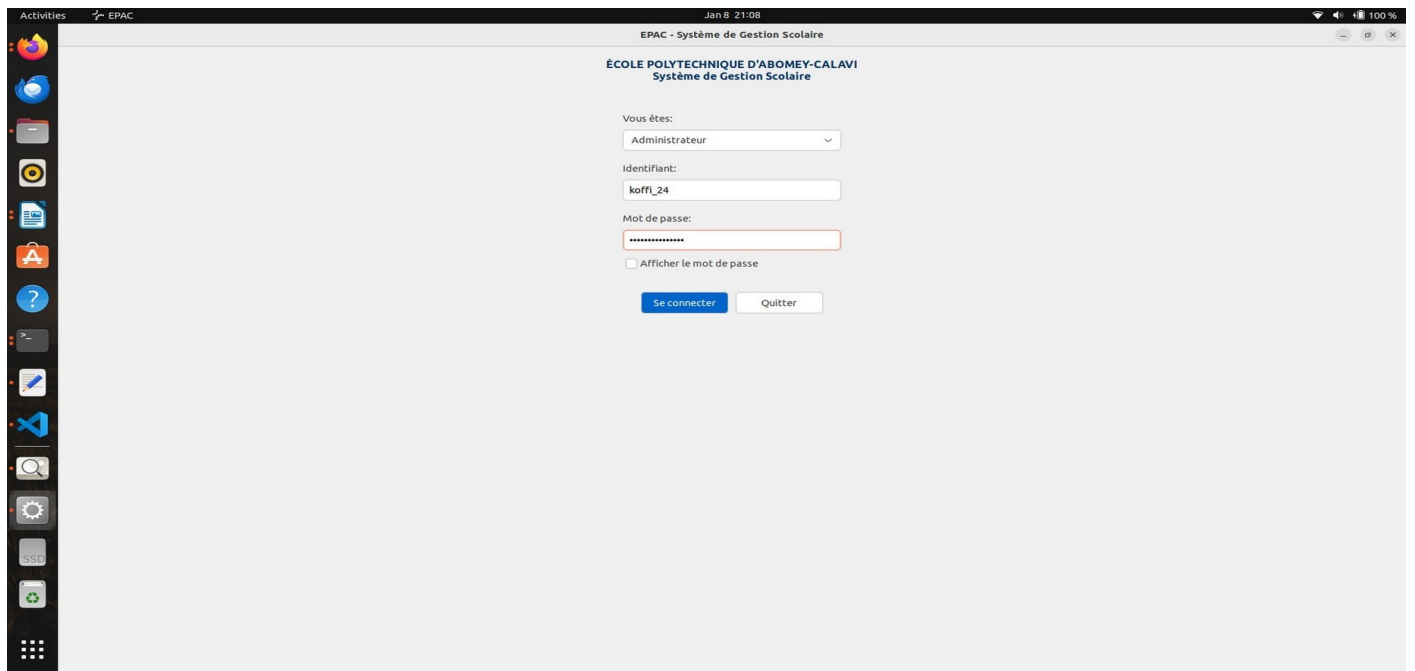


Fig9: LoginFrame Admin

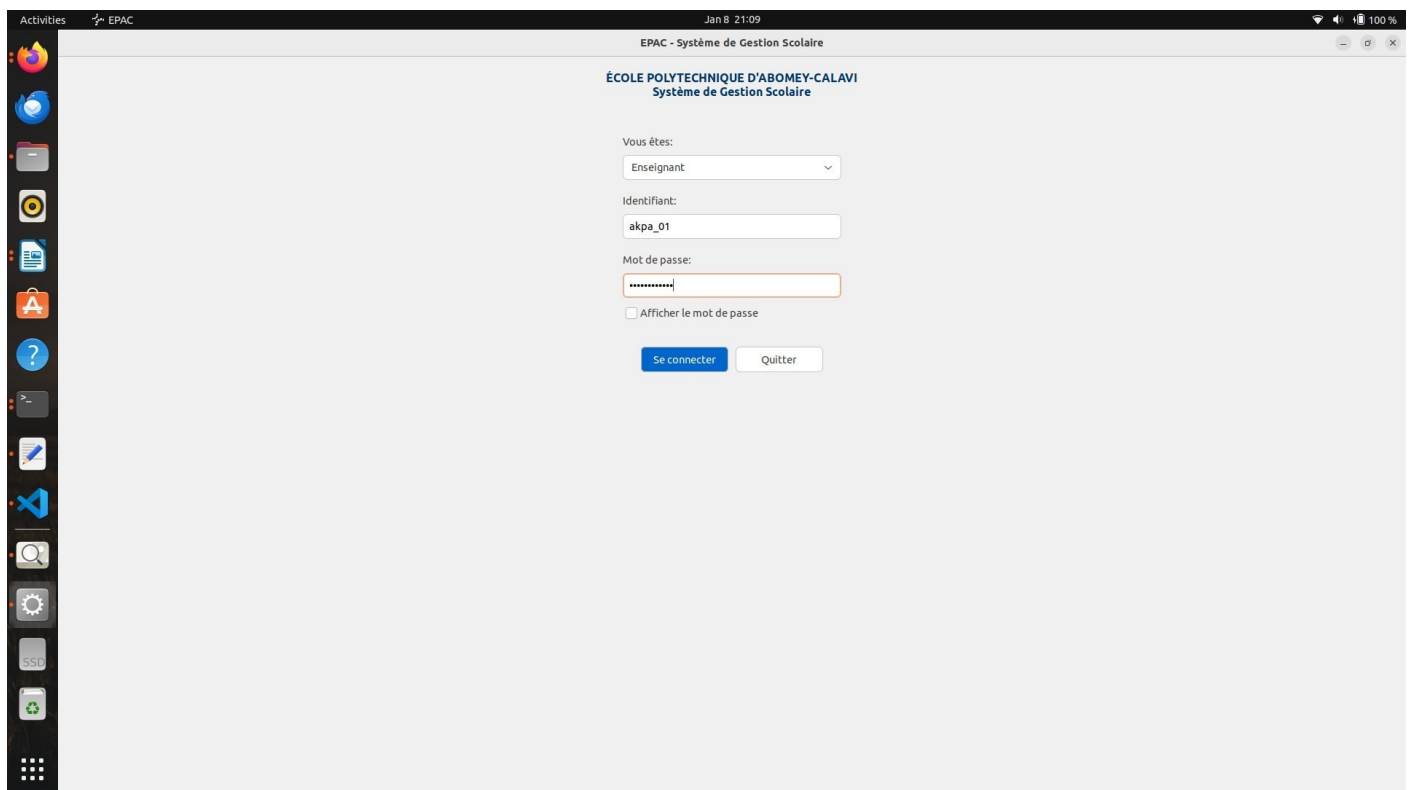


Fig10: LoginFrame Enseignant

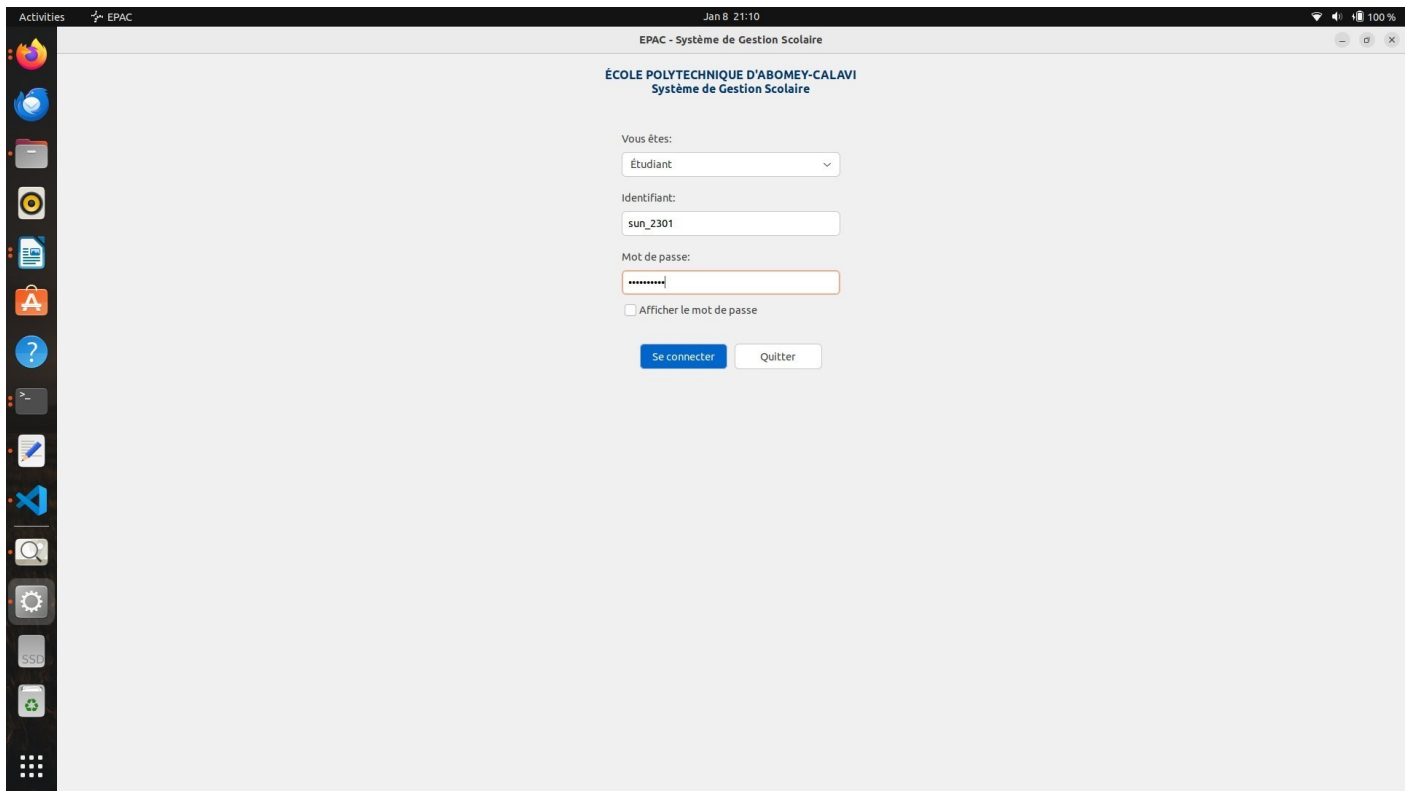
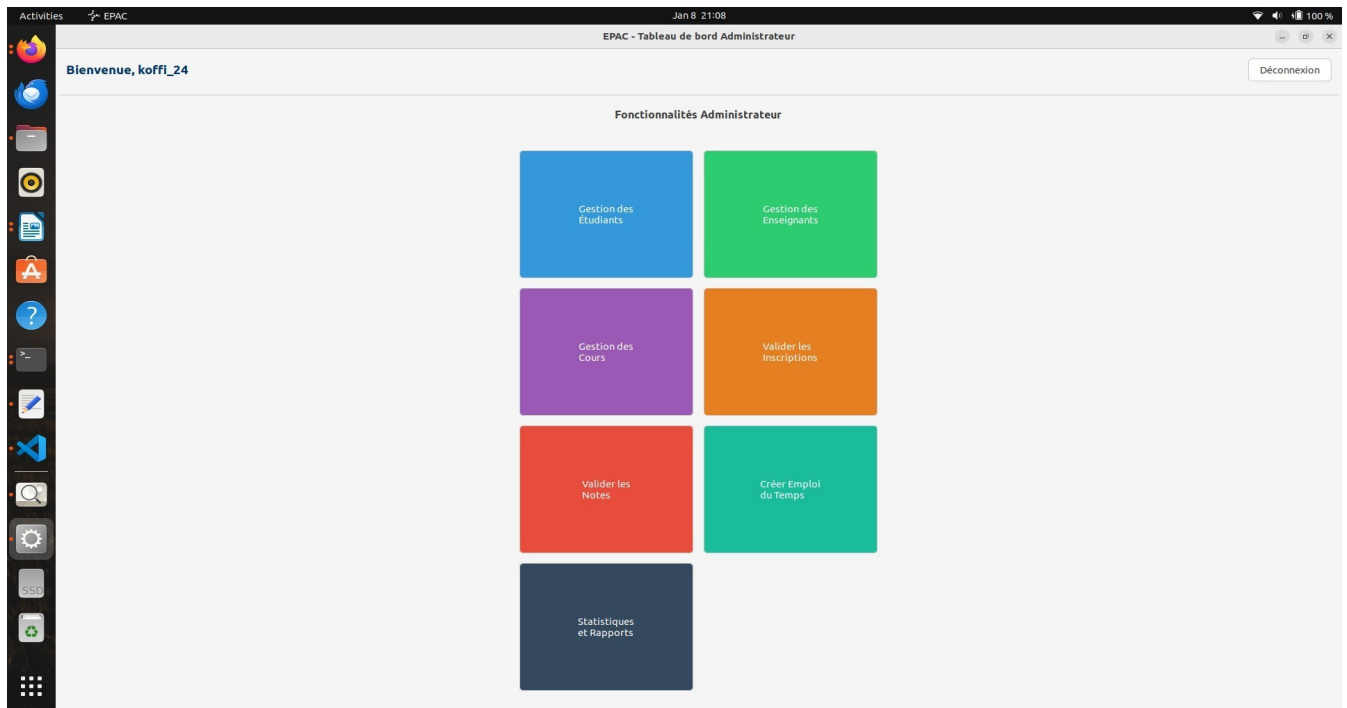


Fig11: LoginFrame Etudiant

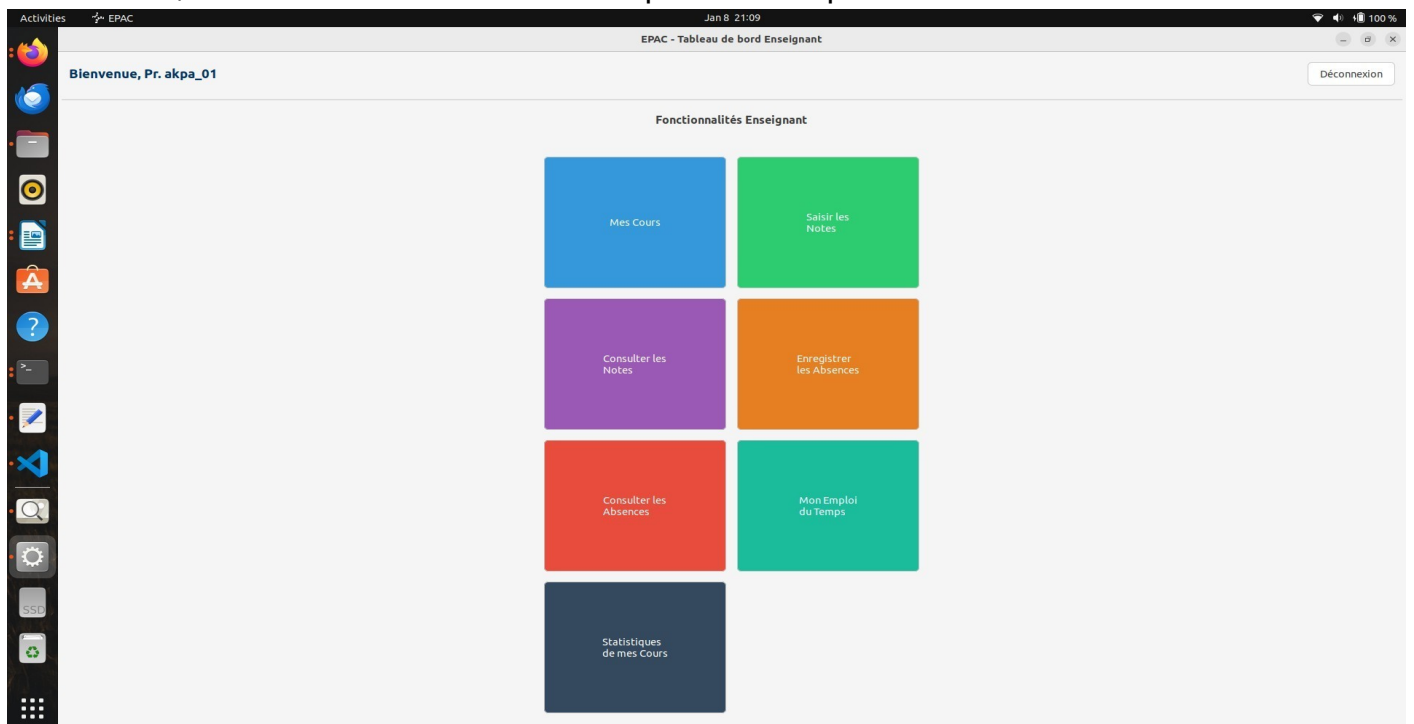
2. Tableaux de bord principaux

Les trois tableaux de bord contextuels sont implémentés avec leur interface complète:

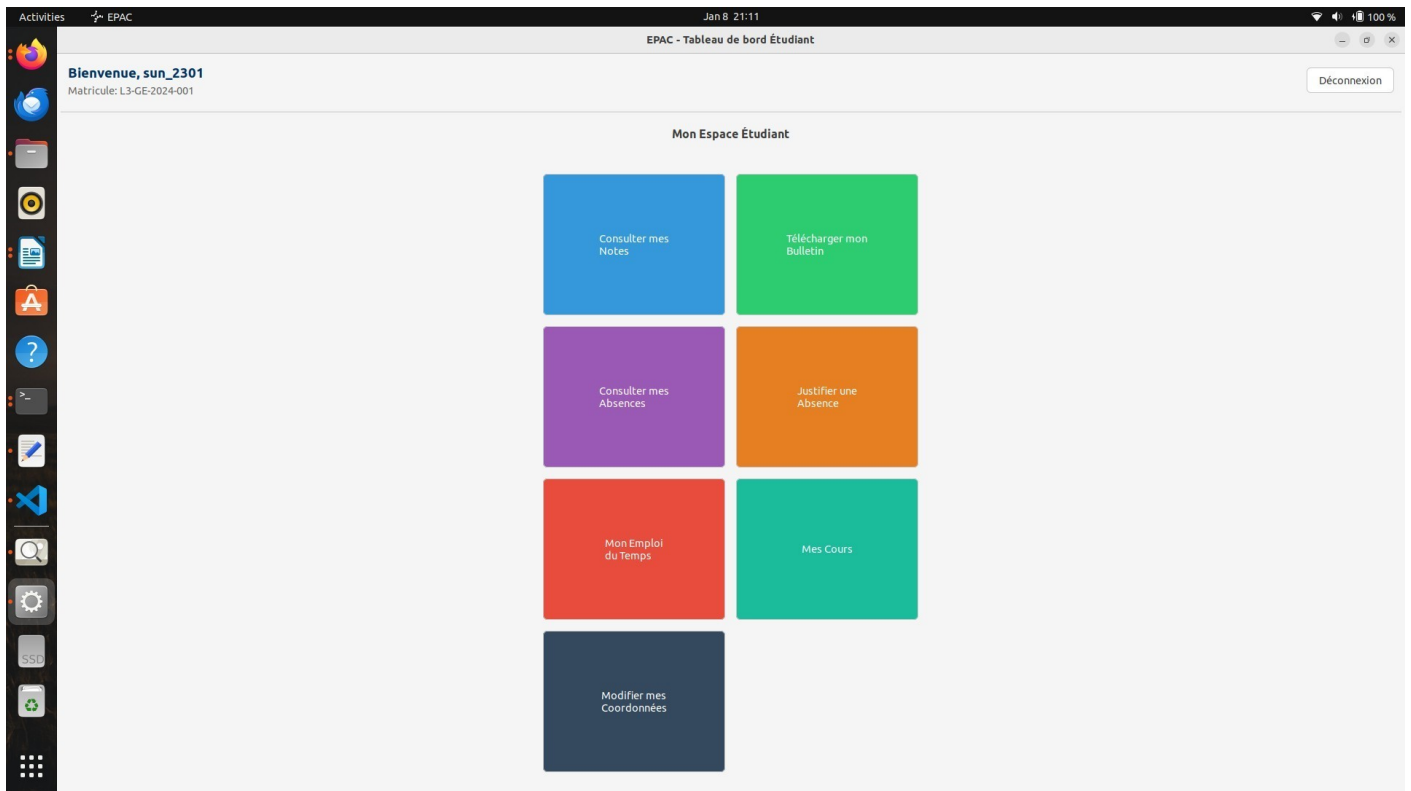
AdminFrame: Interface avec accès aux 7 modules de gestion (étudiants, enseignants, cours, inscriptions, notes, emploi du temps, statistiques)



EnseignantFrame: Interface pédagogique avec accès à la saisie de notes, gestion des absences, consultation de cours et emploi du temps



EtudiantFrame: Interface de consultation pour le suivi académique (notes, absences, bulletin, emploi du temps)



Chaque tableau de bord est organisé en grille de boutons colorés pour une navigation intuitive. Les boutons déclenchent actuellement des messages informatifs indiquant "À implémenter", permettant de valider la structure navigationnelle avant l'implémentation complète des fonctionnalités.

VII.3 Description détaillée du scénario implémenté

Objectif du scénario

Le but est de émontrer le flux complet de gestion des notes en temps réel, impliquant les trois acteurs principaux du système (Enseignant, Administrateur, Étudiant) dans un workflow cohérent et traçable, conforme aux cas d'utilisation CU-02 (Saisir les notes), CU-03 (Valider les notes) et CU-06 (Consulter notes en temps réel).

Acteurs impliqués

- Enseignant : Saisit les notes (CC, TP, Examen) avec possibilité de sauvegarde progressive
- Administrateur : Valide ou rejette les notes soumises en fournissant des commentaires si nécessaire
- Étudiant : Consulte ses notes en temps réel dès leur saisie par l'enseignant

Préconditions

- Les étudiants sont inscrits dans le système (données de démonstration : 10 étudiants de niveau L3-GE)
- L'enseignant est affecté à au moins un cours (exemple : INF301 - Programmation Orientée Objet)
- Les modalités d'évaluation sont définies : CC 30%, TP 20%, Examen 50%
- La couche Database contient les données de démonstration nécessaires

Déroulement du scénario

Étape 1 : Connexion de l'enseignant

L'enseignant lance l'application et accède à l'écran de connexion (LoginFrame). Il sélectionne le rôle "Enseignant" dans la liste déroulante, saisit son identifiant et son mot de passe, puis clique sur "Connexion". Le système vérifie les identifiants et, en cas de succès, affiche le tableau de bord enseignant (EnseignantFrame).

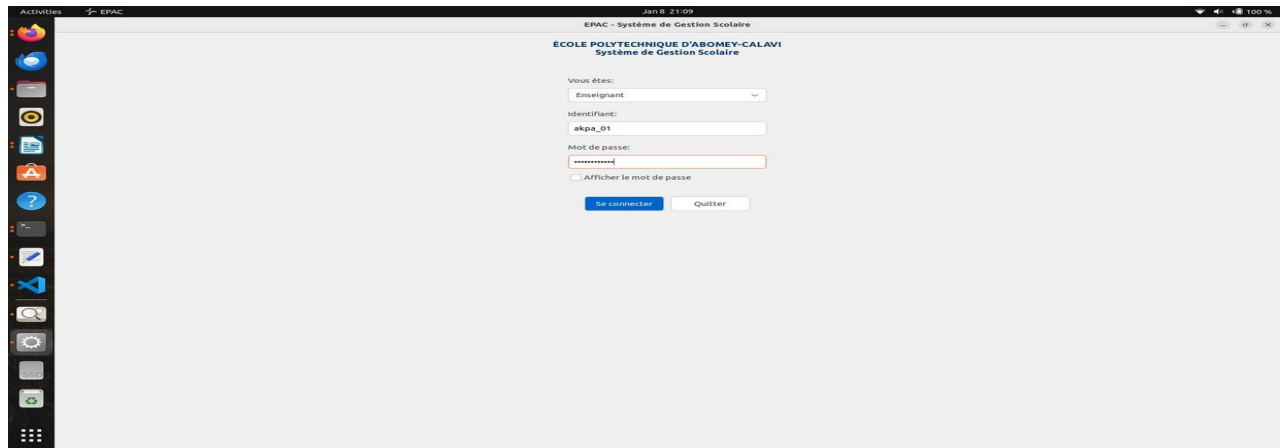


Fig12: LoginFrame Enseignant

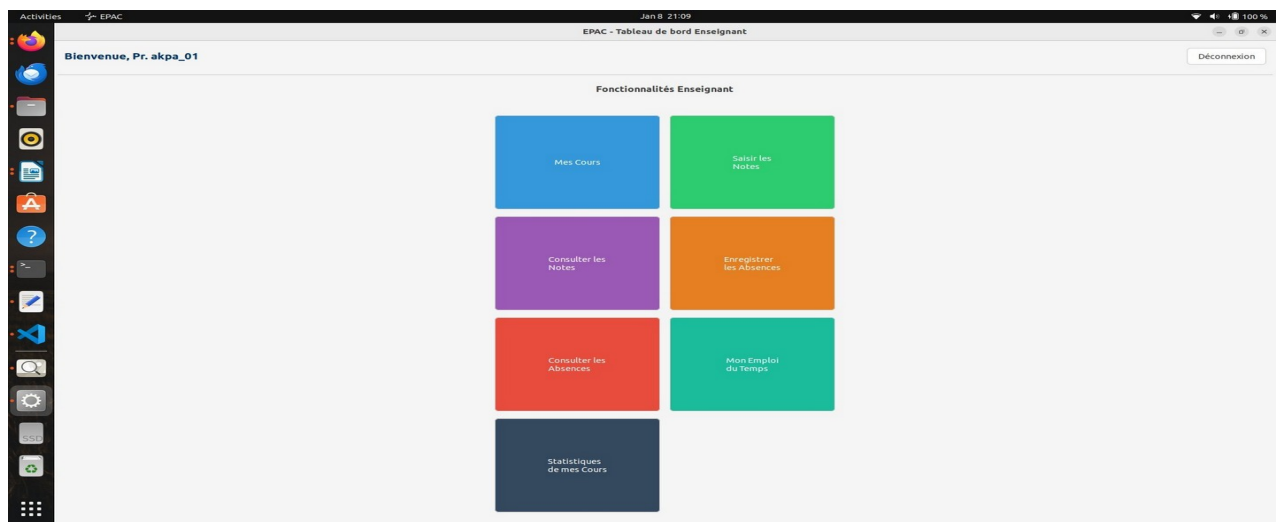


Fig13: EnseignantFrame

Étape 2 : Saisie des notes par l'enseignant

L'enseignant clique sur le bouton "Saisir les notes". Le système ouvre le dialogue de saisie (SaisieNotesDialog) qui affiche la liste déroulante des cours assignés. L'enseignant sélectionne un cours, par exemple INF301 - Programmation Orientée Objet.

Le système charge automatiquement la liste des étudiants inscrits au cours et affiche une grille avec les colonnes suivantes : Matricule, Nom, Prénom, CC(/20), TP(/20), Examen(/20), Moyenne.

À chaque saisie de note, le système effectue les opérations suivantes :

- Validation que la note est comprise entre 0 et 20
- Calcul automatique immédiat de la moyenne : $Moyenne = (CC \times 0.30) + (TP \times 0.20) + (Examen \times 0.50)$
- Attribution de la mention correspondante (Très Bien si ≥ 16 , Bien si ≥ 14 , Assez Bien si ≥ 12 , Passable si ≥ 10 , Ajourné si < 10)
- Colorisation visuelle de la cellule de moyenne selon la performance
- Enregistrement immédiat avec le statut "Disponible"

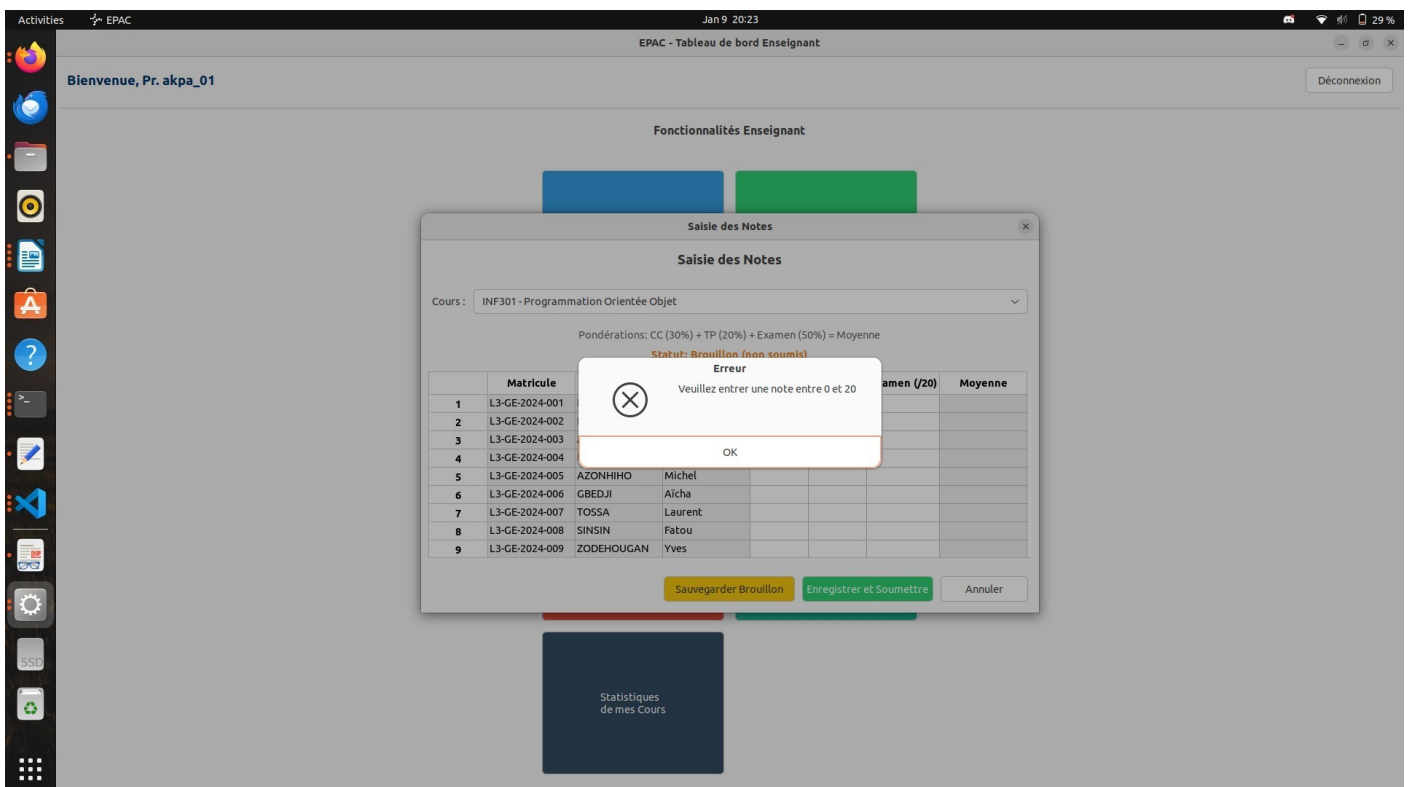


Fig14: xemple de gestion d'erreur de mauvaise note entrée

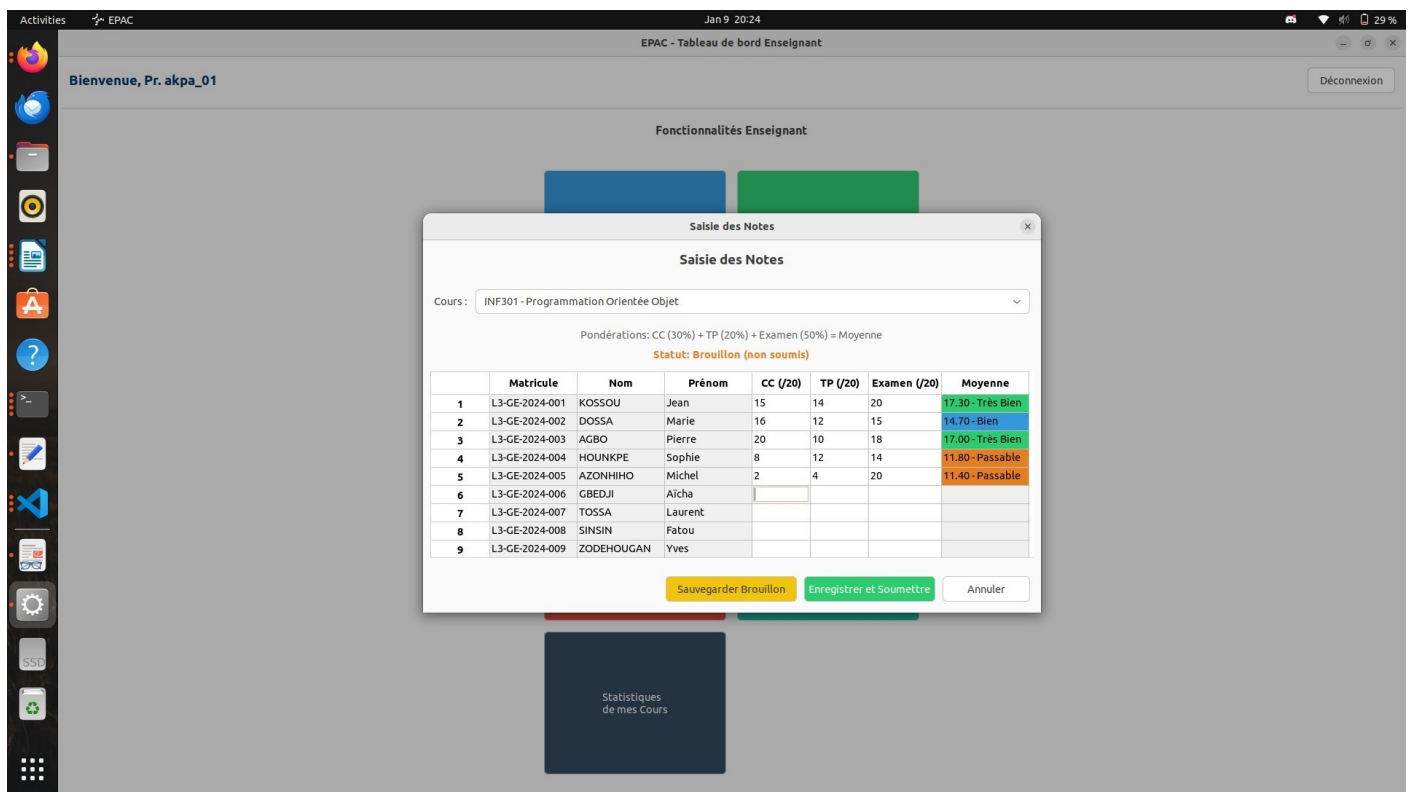


Fig15: Saisie de notes

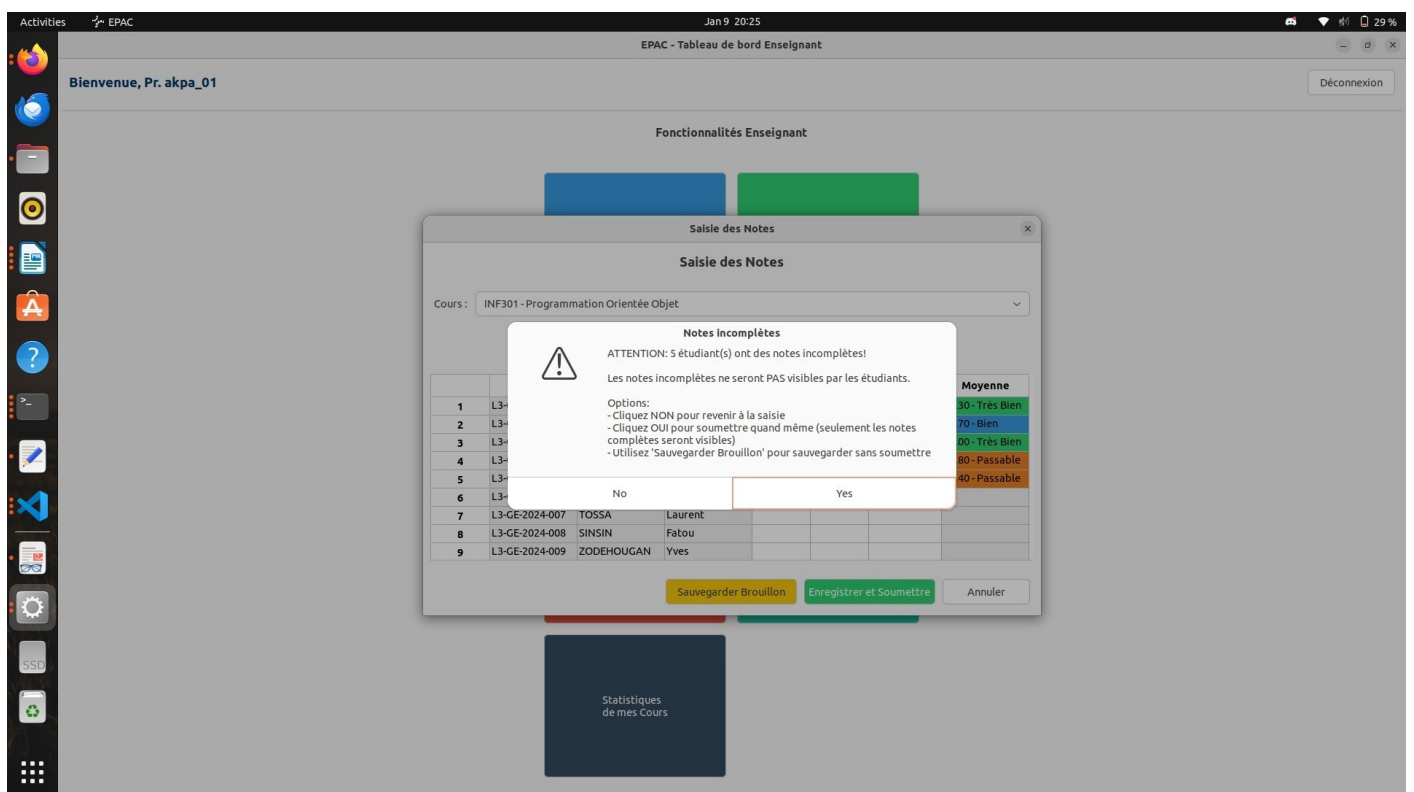


Fig16: Tentative de soumission de notes incomplètes

Étape 4 : Sauvegarde progressive

À tout moment durant la saisie, l'enseignant peut cliquer sur le bouton "Sauvegarder Brouillon". Cette fonctionnalité permet d'enregistrer les notes partielles, même si tous les étudiants n'ont pas encore de notes complètes. Le système enregistre les notes partielles qui restent visibles en temps réel par les étudiants concernés.

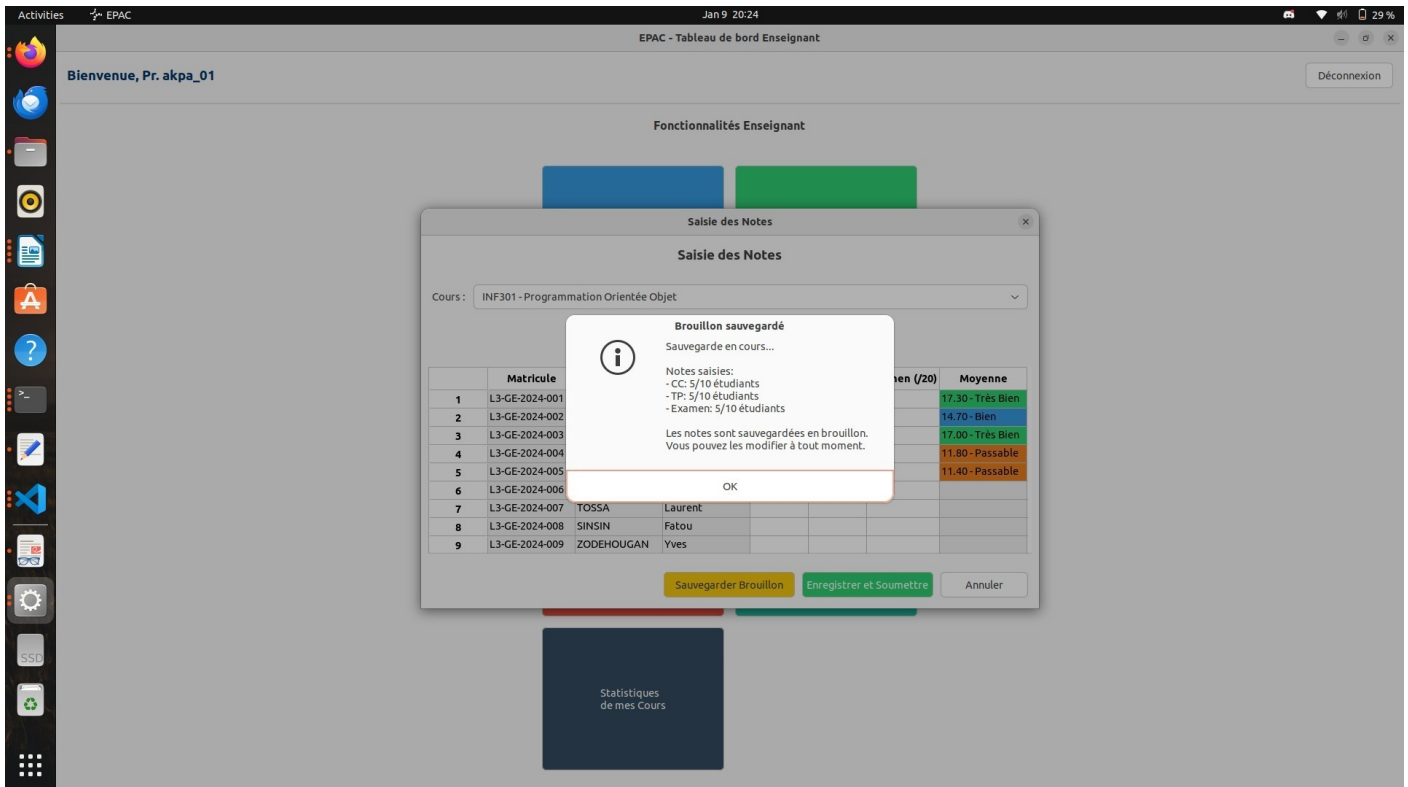


Fig17: Sauvegarde de brouillon

Étape 5 : Soumission des notes par l'enseignant

Une fois la saisie terminée pour tous les étudiants du cours, l'enseignant clique sur le bouton "Enregistrer et Soumettre".

Le système effectue une vérification stricte : toutes les notes doivent être complètes (CC, TP et Examen saisis) pour chaque étudiant. Si des notes sont incomplètes, le système refuse la soumission. Si toutes les notes sont complètes, le système change le statut à "Soumise" et affiche un message de confirmation.

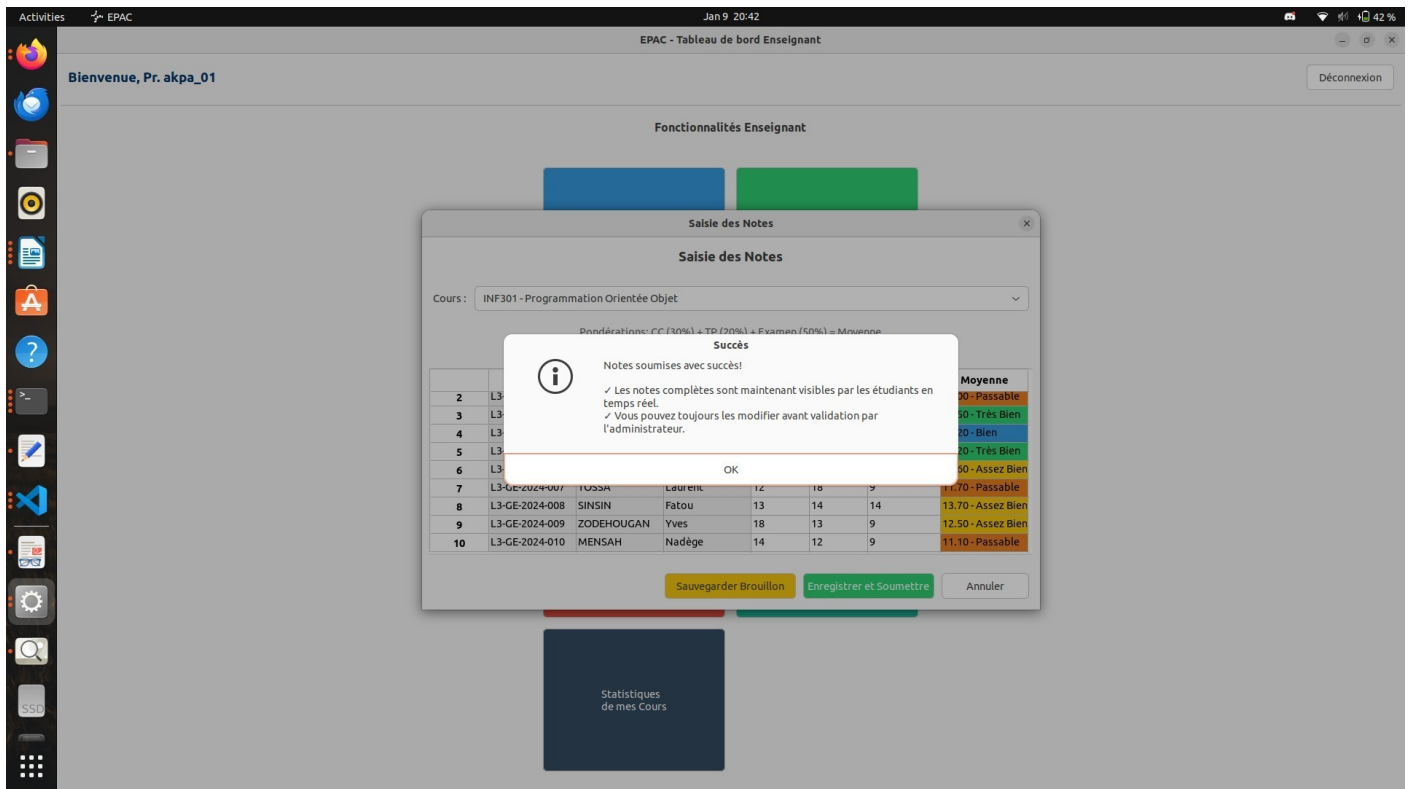
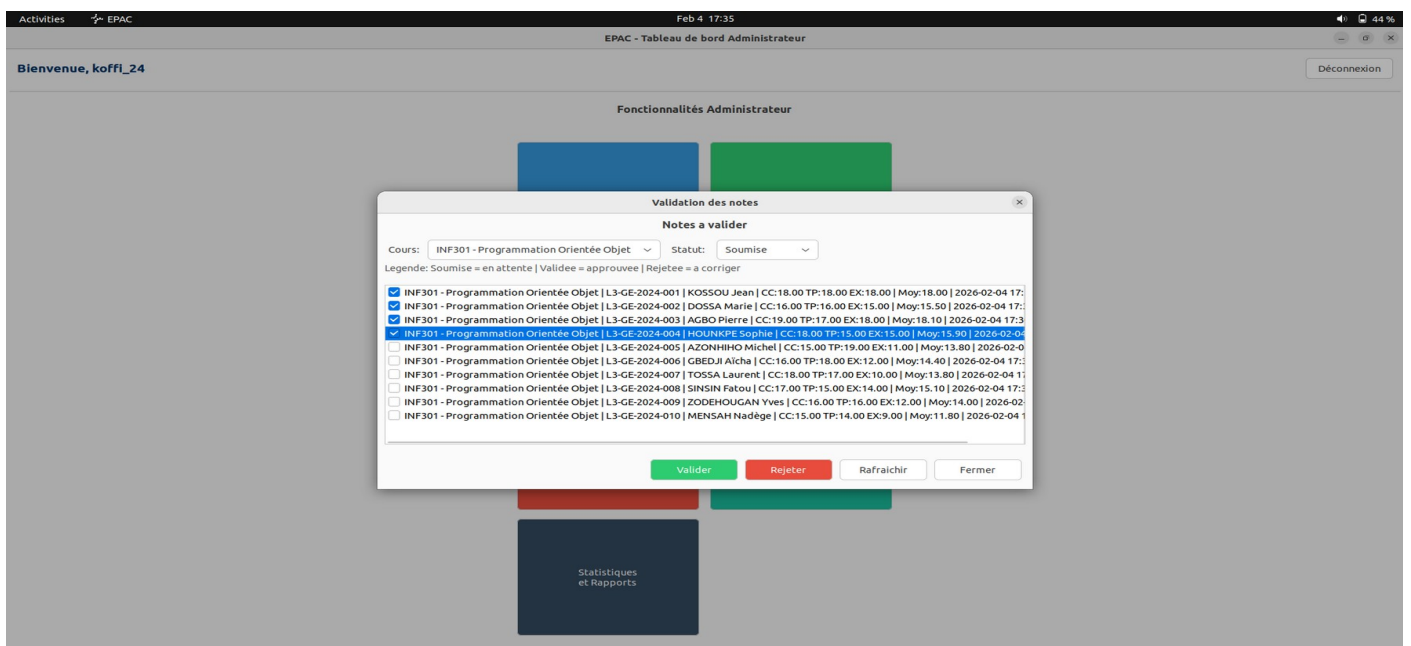


Fig18:Soumission de notes réussie

Étape 6 : Validation par l'administrateur

L'administrateur se connecte au système et clique sur "Valider les notes". Le système ouvre le dialogue de validation (ValidationNotesDialog).



L'administrateur dispose de deux options :

Option A - Valider les notes : L'administrateur coche les notes qu'il souhaite valider et clique sur le bouton "Valider". Le système change le statut à "Validée", enregistre la date de validation, et verrouille les notes de manière définitive.

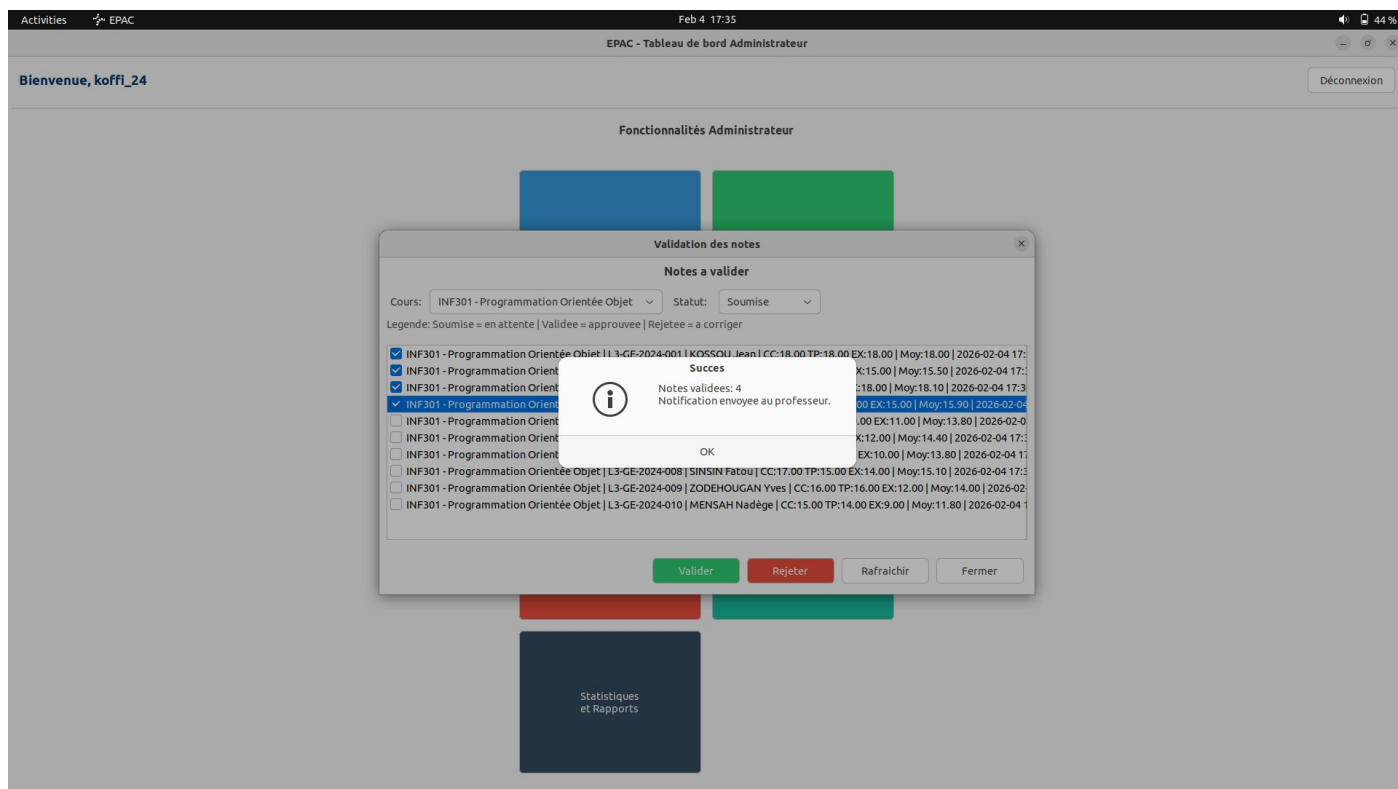


Fig20: Validation des notes

Option B - Rejeter les notes : Si l'administrateur détecte une anomalie, il coche les notes concernées et clique sur le bouton "Rejeter". Le système demande obligatoirement un motif de rejet. L'administrateur entre un commentaire explicatif et le système change le statut à "Rejetée".

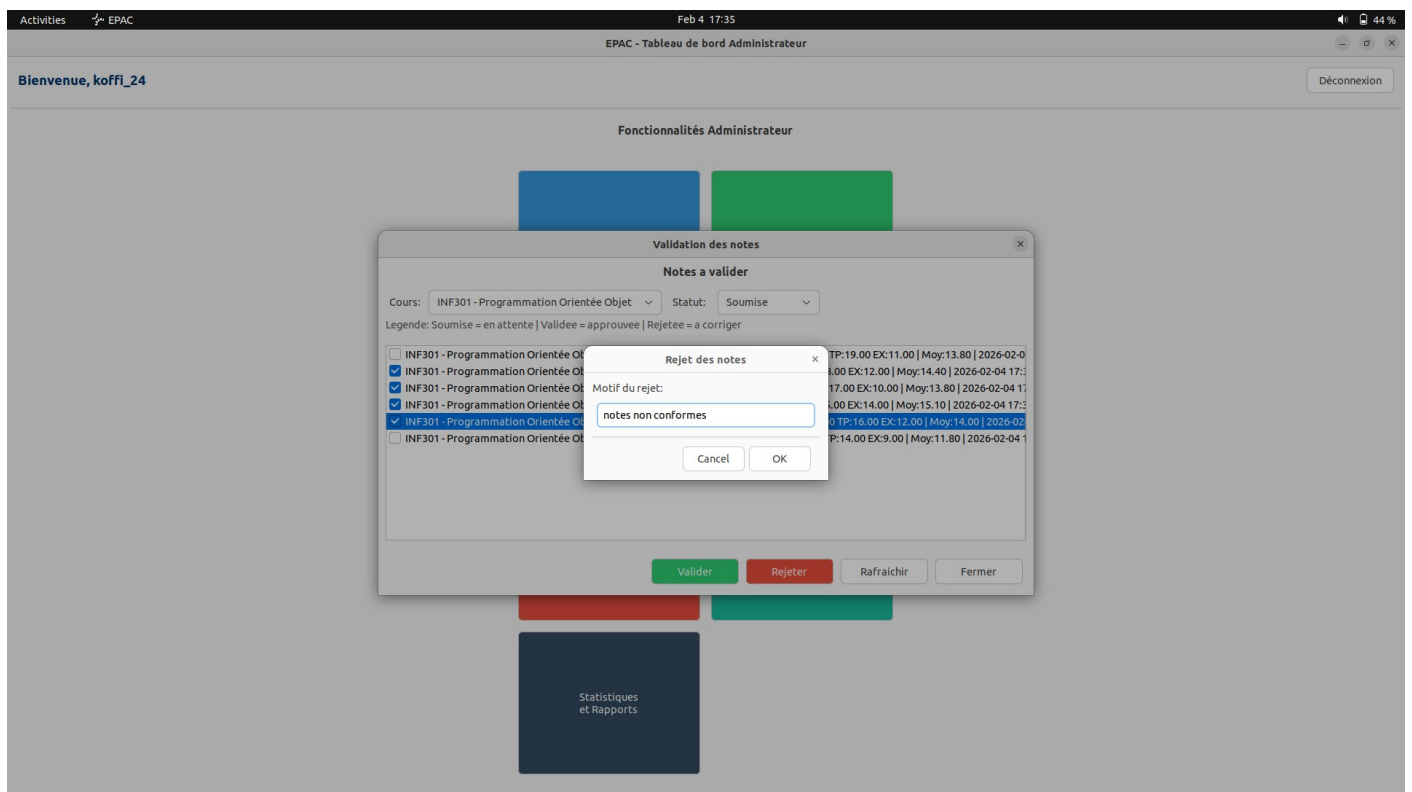


Fig21: Rejet de notes: indication du motif du rejet

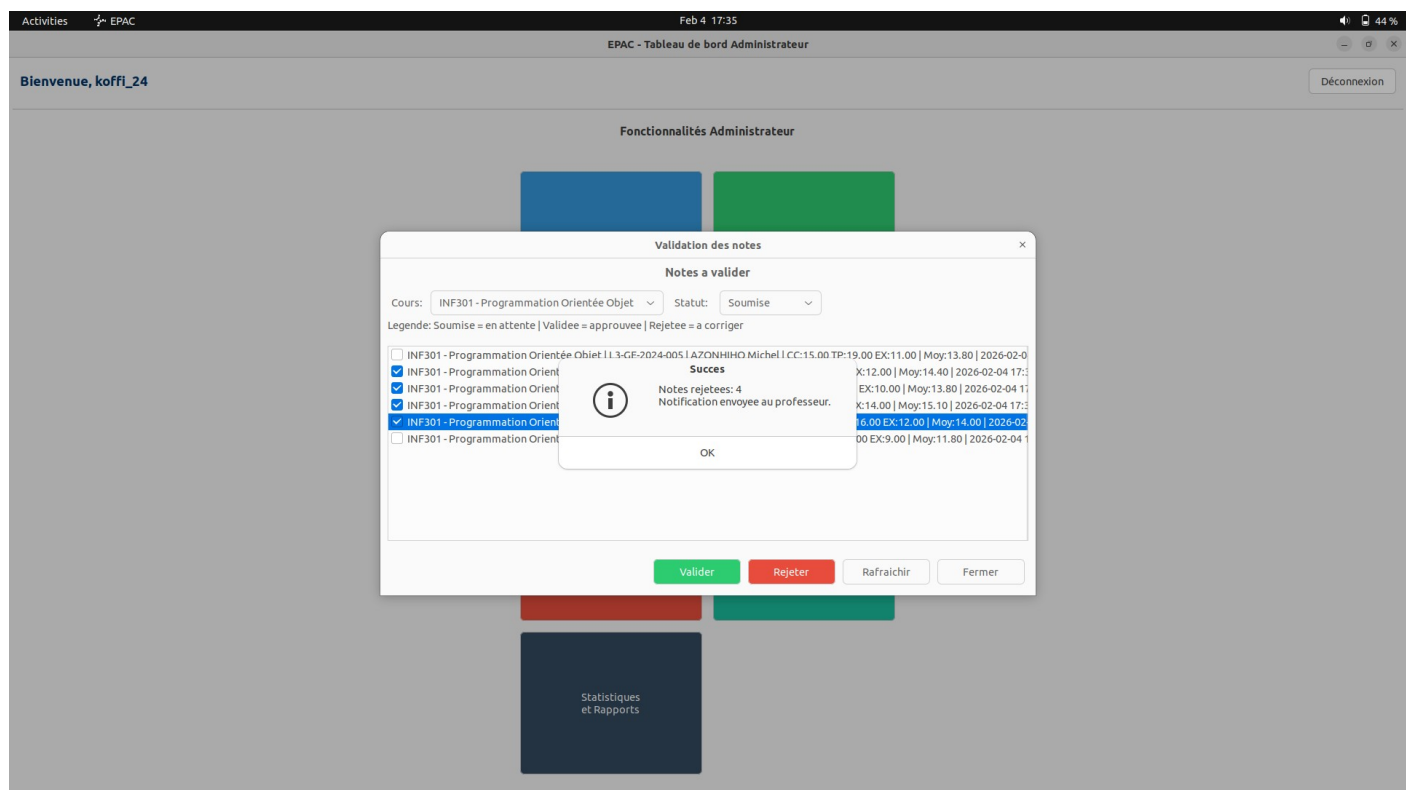


Fig22: Rejet de notes réussi.

Étape 3 : Consultation en temps réel par l'étudiant

Pendant que l'enseignant poursuit la saisie ou après soumission et/ou validation des notes, un étudiant lance l'application et accède à l'écran de connexion. Il sélectionne le rôle "Étudiant" et entre son matricule, par exemple L3-GE-2024-001.

Le système valide l'existence du matricule dans la base de données et affiche un message de bienvenue personnalisé : "Bienvenue, KOSSOU Jean". Le tableau de bord étudiant (EtudiantFrame) s'affiche.

L'étudiant clique sur "Consulter mes notes". Le système ouvre le dialogue de consultation (ConsultationNotesDialog) qui affiche immédiatement les notes déjà saisies par l'enseignant et/ou déjà validées, sans aucune attente de fin de semestre.

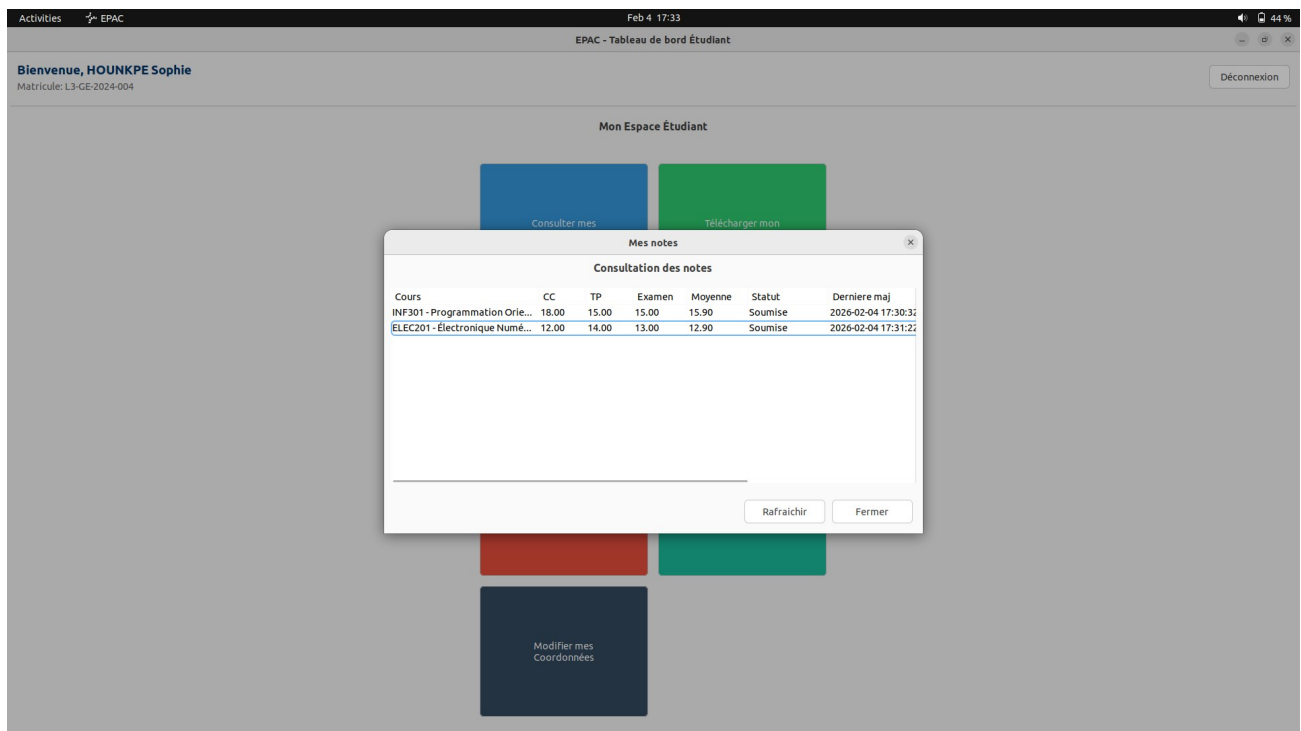


Fig23: Consultation de notes par l'étudiant.

Postconditions

À l'issue de ce scénario :

- Les notes sont enregistrées dans le système avec une traçabilité complète
- Les moyennes sont calculées automatiquement selon les pondérations définies

- Les étudiants ont un accès en temps réel à leurs notes, sans attendre la fin du semestre
- L'administrateur a validé ou rejeté les notes avec des commentaires si nécessaire
- L'historique complet est conservé (updatedAt, submittedAt, validationAt, adminComment)
- Les notes validées sont verrouillées et ne peuvent plus être modifiées

Ce dialogue démontre la faisabilité technique et valide l'approche adoptée pour les autres fonctionnalités.

7.3 Architecture logicielle en place

Structure du projet:

GestionScolaire/

```

├─ src/
|   ├─ frames/      # Fenêtres principales (Login, Admin, Enseignant, Etudiant)
|   ├─ dialogs/     # Boîtes de dialogue fonctionnelles
|   ├─ models/      # Classes métier (préparées pour implémentation)
|   └─ database/    # Couche d'accès aux données
├─ include/         # Fichiers d'en-têtes
├─ bin/             # Exécutable et fichiers compilés
├─ Makefile         # Système de build automatisé
└─ README.md        # Documentation du projet

```

Cette architecture modulaire facilite le développement incrémental et la collaboration. Le code source est versionné sur GitHub, accessible [via ce lien](#) pour assurer la traçabilité et permettre le suivi de l'évolution du projet

7.4 Technologies et outils utilisés

Stack technique:

Langage: C++11

Bibliothèque GUI: wxWidgets 3.0.5

Compilateur: g++ (GNU Compiler Collection)

Système de build: Makefile personnalisé

Gestion de version: Git avec dépôt GitHub

Plateforme de développement: Ubuntu 22.04

Base de données prévue: SQLite3 (léger, portable, sans serveur)

VIII- CONCLUSION

Ce projet illustre la démarche complète de conception et développement d'un système informatique, de l'analyse des besoins à l'implémentation technique. La méthodologie UML nous a permis de structurer notre réflexion et de produire une base solide pour le développement.

L'implémentation du système a franchi une étape décisive avec la mise en place d'une architecture en trois couches strictement séparées. Le refactoring complet effectué a permis de déplacer toute la logique métier des interfaces graphiques vers les modèles, garantissant ainsi la maintenabilité et l'évolutivité du code. Le scénario de gestion des notes (saisie, validation, consultation en temps réel) est pleinement opérationnel et démontre la robustesse de l'architecture choisie. Les méthodes statiques implémentées dans les classes métier (Utilisateur, Note, Absence, Etudiant) encapsulent efficacement la logique et offrent une API claire pour l'interface graphique.

Ce projet continue d'évoluer et représente une base excellente pour un système de gestion scolaire moderne et efficace, contribuant à la digitalisation des processus académiques à l'EPAC.

Le code source, accessible publiquement sur GitHub, témoigne de notre engagement envers la transparence et les bonnes pratiques de développement logiciel.